

Contents

1	Introduction	2
1.1	Multiple sequence alignment	4
1.1.1	List of used MSA algorithms	5
1.2	Structural alignments	5
1.2.1	Distance matrix alignment method	7
1.3	MSA set representations	8
1.4	Sum-of-pairs-score	9
1.5	Jaccard distance	10
1.6	Classification and accuracy measurement	10
2	Methods	14
2.1	Set up and protein database	14
2.2	Data availability and data bias	15
2.3	General programmatic procedure and input data	17
2.4	Scoring	21
3	Results and discussion	27
3.1	Scoring MSA algorithms with HomFamRef	27
3.2	Measuring specificity and overall uncertainty	36
4	Conclusion	37
5	Side results	39
5.1	The Hamming distance defined in terms of our set representations	39
5.2	A small conjecture about the uniqueness of the PSTAR reference set representation	40
5.3	A metric space involving three sets and a corollary	40
5.4	An unproven metric involving three sets	43

Abstract

Multiple sequence alignment (MSA) plays an important role in evolutionary genetics in that it helps to find homologue subsequences in a number of sequences that carry genetic information, such as DNA, RNA or amino acid chains. However, it has not been possible to find an algorithm that produces exact solutions for an MSA problem in polynomial time. That is why over the years many different heuristic MSA algorithms have been developed to try to give good estimate solutions that bear biological meaning in a reasonable time. Mainly in an effort of manual labour many different reference data sets have been defined in order to score and compare different heuristic MSA algorithms with each other. These data sets tend to be small, hand-curated and their creation process and validity is mostly intransparent. This work aims to combat these problems by using protein pairwise structural alignments as references, both extending the data availability for the reference, as well as making their creation process transparent, ultimately aiming to create a more generalized reference and strengthening confidence in MSA algorithm ranking quality. We developed a software that scores and compares MSA quality based on real experimental protein structure data from the "Protein Data Bank" (PDB) and introduce novel measure indices to rank different MSA algorithms. In doing so we gained confidence that HOMSTRAD is a good, albeit not perfect reference for protein MSA that mainly suffers from low data availability. We also criticize the widespread use of the sum-of-pairs-score (SPS) as a solitary measure index for MSA quality and propose alternative measures such as the Jaccard index. The final MSA algorithm ranking differs from that of pre-existing literature in that it scores FAMSA lower than both MAFFT-sparsecore and MUSCLE. We find that MUSCLE and learnMSA2 perform best on the HomFam dataset according to our pairwise reference structure alignment references created with DALI.

1 Introduction

An MSA algorithm is an optimization algorithm that takes a list of three or more input sequences and inserts gaps into each sequence, such that all resulting gapped sequences conform to the same length, which is called the alignment length. This is done while maximizing the sum of residue similarities between all pairs of sequences, i.e. maximizing the sum-of-pairs-score (SPS) [1]. Similarity measures between pairs of sequences and pairs of residues can differ based on context. In a biological context the sequences are mostly representing DNA, RNA and protein chains and a good MSA can be used to find homologous regions between these molecules. Two popular examples of these similarity measures,

which are called scoring schemes in the case of protein MSAs are *PAM* and *BLOSUM* [2], which simply assign a score to each pair of amino acid residues. These residue scores are based on biological and biochemical findings and there is currently no unanimous best way of scoring residue pairs. Finding an optimal solution to the MSA problem with SPS in polynomial time is unlikely, since most common instances of the problem are proven to be NP-hard [3][4]. That is why often heuristic algorithms are being used to estimate the solution that best represents the true evolutionary process. It is important to note that the biologically most meaningful solution does not necessarily equate to the mathematically optimal solution, mostly because the associated similarity measures in the optimization are lacking in that they fail to accurately describe all of the complex parameters that actually define "evolutionary closeness". Unfortunately this means that verifying or measuring the quality of the solution obtained by these algorithms requires in itself an estimate solution, which we will be calling reference alignments. One of the most prominent of these reference alignments is the HOMSTRAD database [5]. Due to the hand-curated nature of the database however, it is not clear at all why or if the alignments contained within are sufficiently reflecting reality or if the sample meets the requirements to allow the use of methods of inferential statistical analysis, i.e. whether or not heuristic MSA algorithms that perform well on this sample will also perform well in general. The MSAs for each HOMSTRAD homology family are structure-based and "derived using COMPARER" [6][7] and MNYFIT [8] and "carefully examined by eye" [5]. Nowadays the best canon argument for using it seems to be that "HOMSTRAD encompasses expertly curated structural alignments of homologous proteins within 1032 protein families" [9]. The state-of-the-art-methods of MSA benchmarking and scoring use these "gold-standard" databases of protein alignments like HOMSTRAD or BALIBASE [10] as ground truth to measure sensitivity via popular scores like a version of the SPS that uses a reference alignment to score single residue pairs and the TC-score. The results are then used to create rankings of the tested MSA algorithms [11][12]. Here we describe a possible alternative way to measure the accuracy of MSA algorithms by utilizing pairwise structural alignments of the involved proteins as reference, and measure MSA quality with the PSTAR-score, a novel scoring index which is similar to the sum-of-pairs score, as well as with the Jaccard index, which considers specificity. We also provide a software implementation, called PSTAR-MSA (Pairwise STructural Alignment Reference-MSA), which can be found at <https://github.com/MarGabor/PSTAR-MSA>. The main goal of this work is to provide an additional measure to either verify the integrity of or uncover possible inconsistencies within the most commonly used reference databases such as HOMSTRAD, as well as generalizing the process of MSA benchmarking by having access to a larger data set (PDB), which might even be expandable by

the relatively recent addition of artificial intelligence protein structure predictors like *AlphaFold* [13].

1.1 Multiple sequence alignment

Definition 1. : *Multiple sequence alignment (MSA)*

Given an alphabet Σ and a list of N sequences $S = (S_1, \dots, S_i, \dots, S_N)$ with $S_i = (s_{i1}, \dots, s_{ij}, \dots, s_{im_i}) \in \Sigma^{m_i}$, then $S' = (S'_1, \dots, S'_i, \dots, S'_N)$ with $S'_i = (s'_{i1}, \dots, s'_{ij}, \dots, s'_{iM}) \in (\Sigma \cup \{-\})^M$ and a gap symbol $-$, is called a **multiple sequence alignment** of length M for sequences S , if S is equal to S' , when removing all gaps from each S'_i . An algorithm that generates a **multiple sequence alignment** from an input S is called a **multiple sequence alignment algorithm**.

In a biological context, the sequences mostly represent genetic information, such as RNA, DNA, or amino acid chains of proteins. This biological context allows us to interpret the MSA calculation as part of the process of finding homologous sites among the input sequences. Homologous sites are parts of genetic sequences, which are inherited from a common ancestor. Ideally, if there is high degree of evolutionary closeness between subsequences, then this should result in these subsequences being aligned in an MSA [1]. Thus, a good MSA algorithm is reliable in a way that the MSA it produces from any given list of sequences can be used to identify similar sites among input sequences and a good scientist is responsible for choosing a list of input sequences together with an appropriate scoring matrix and is then able to infer evolutionary relationships from that. This is partly due to analogous sites, as opposed to homologous sites, which are sites that can appear similar to each other, depending on how exactly similarity is measured, but are of no (close) common descent. These sites are the result of evolutionary convergence or of chance. Evolutionary convergence happens where similar selective pressures required organisms of different descent to have similar phenotypical properties in order to pass on their genes. Chance happens when a region is not really important for survival or reproduction. Thus, it is important to be critical when judging evolutionary closeness on the basis of MSAs [14]. Because telling analogous and homologous regions apart from another is still an ongoing and difficult task [15][16]. In our case, homology mostly equates to similarity, since our input data set HOMSTRAD (or HomFam) is divided into homology groups. As the name implies, these protein sequences are taken from families of organisms known to be related. However, when expanding the approach to bigger and more general data set, other measures will have to be put in place to ensure the exclusion of similar, but non-homologous input sequences.

1.1.1 List of used MSA algorithms

CLUSTALO [17] follows the progressive alignment heuristic [18], which uses pairwise sequence alignment to identify the most similar pair among all sequence pairs and then proceeds to use this pairwise alignment as a seed alignment to align the rest of the sequences to it. The main advantage of progressive alignment is its speed, allowing the alignment of a lot of sequences in little time. However, errors that occur during the extension of the seed alignment are propagated throughout the alignment making the most basic approach of it unreliable. Modern aligners like *CLUSTALO* modified this approach further to limit the cumulated error and also increase speed.

MUSCLE [19] belongs to a subclass of progressive aligners called iterative aligners. The iterative approach minimizes dependence of the output on the order of the input sequences, because other than the progressive alignment methods in general, iterative aligners constantly realign the initial seed alignments.

MAFFT-sparsecore [20] is a progressive aligner that utilizes a Fast-Fourier-Transform (FFT) to generate its alignments. After progressive alignment is done, it follows up on it with an iterative refinement step. However, this refinement step is not the same as constantly realigning seed alignments during the actual MSA generation.

FAMSA [21] is a progressive aligner that aims to improve alignment speed, while maintaining a comparable accuracy as other progressive aligners such as *CLUSTAL* for large datasets.

T-COFFEE [22] is another progressive aligner that aims to consider the special problem constraints of MSA more, which is not only the pairwise dependence of sequences upon each other, but also the dependence of any other sequence on either of the first two sequences. This is done by a library extension of additional pairwise alignments which is consulted in each step of the progressive alignment.

learnMSA is different from all of the above aligners in that it utilizes a "statistical learning approach of profile hidden Markov models" [23] and a neural network to generate alignments.

learnMSA2 [24] is a modified and extended version of *learnMSA*.

1.2 Structural alignments

Protein structural alignment seeks to find structural similarities between the tertiary structure of proteins. Formulated as an optimization problem, it aims to maximize some spatial similarity measure between the tertiary structures of proteins and returns at the very least a set of residue pairs representing a maximal similar substructure [25]. Other than with MSA, it is not necessarily possible

to infer evolutionary relationships based on similarity of two or more structures. This is due to evolutionary functional convergence, where the proteins of species of sufficiently different origin, but which were subjected to the same or similar selective pressures, were guided by evolution to fulfill a similar function. Often in proteins, a similar function is the result of a similar structure, but there are many examples of proteins of similar structure that share almost no common sequences [26]. As a result, when using pairwise structural alignments as a reference, as we are doing here, it is important to bear in mind that an MSA can get high scores despite of vastly **dissimilar** sequences. This is a double-edged sword, because certain structure motifs are also often a result of only a few functional amino acids, abridged with longer stretches of functionally unimportant residues, i.e. "padding material". Point mutations of these unimportant residues will have no significant impact on the evolutionary fitness of the given organism. Hence, structural alignment also allows us to uncover very distant evolutionary relationships that otherwise would have been very difficult to see from sequence data alone. When the sequences differ vastly, but the corresponding structures are somewhat similar, then this might be indicative of a distant relatedness or functional convergent evolution. This means that an alignment can be correct in the sense that the alignment algorithm found a global or local optimal solution with highly similar substructures and it still does not unambiguously imply an evolutionary relatedness. A good scientist is still necessary to tell analogues and homologues apart from another [26]. "Methods for the identification of local structural similarities alone are not sufficient to spot cases of convergent evolution" [27]. The consequences for us will be that when we score the alignment of unrelated, but structurally analogous proteins that we might falsely give a low score, where it is not appropriate, since the structural similarities in that case are higher than any MSA algorithm might reasonably assume based on sequence data alone. Still, on average protein amino acid sequence divergence is correlated with tertiary structure divergence [28] and we assume that the effect of convergent evolution, since it is also assumed to be the exception rather than the rule, is negligible on data sets of statistically relevant size. Additionally, when we score alignments based on references that organize their data into homology groups such as HOMSTRAD (or HomFam for that matter), we can be almost certain that similarity equates to homology. As with MSA, globally optimal or exact solutions for the structural alignment problem are computationally intractable [29]. Searching for an exact solution is also not sensible for other reasons, since experimental data for protein structure is inexact by default, due to the introduction of noise during the physical measurement process. Polynomial time algorithms for exact solution approximation have been developed. Although still not feasible for everyday use, they can be considered an option, in case that the acquisition of more precise solutions should be of importance. Most common (pairwise)

structure alignment algorithms however, such as DALI [30], are heuristic [25].

1.2.1 Distance matrix alignment method

One popular structural alignment method is the distance matrix alignment (DALI) [30]. It is widely used due to its reliable performance on test sets (Tables 2 and 3 in [31]) as well as its standalone version "DaliLite", which allows programmatic access to DALI alignments. Furthermore, future versions of our software can implement databases of DALI pairwise alignments and the integration of AlphaFold structure predictions with DALI should simplify future extensions of our data basis.

Definition 2. : Metric distance matrix

Given a graph $G = (\{v_1, \dots, v_i, \dots, v_N\}, E)$, a matrix $A = (a_{ij})$ is called a **metric distance matrix**, if $\forall i, j \in \{1, \dots, N\} : d(v_i, v_j) = a_{ij}$ and d is a metric.

The set of vertices in the case of proteins is the set of backbone C_α atoms. Edge weights for the edge (v_i, v_j) are calculated from the spatial coordinates in the PDB files of the respective proteins, using the Euclidian distance $d(v_i, v_j)$. When comparing two structures A, B with nodes $\{v_1^A, \dots, v_i^A, \dots, v_n^A\}$ and $\{v_1^B, \dots, v_i^B, \dots, v_n^B\}$, first two metric distance matrices are calculated for each one via $a_{ij} = d(v_i^A, v_j^A)$ and $b_{ij} = d(v_i^B, v_j^B)$. The DALI algorithm now proceeds to chunk up these matrices into submatrices according to the distance matrices of single hexapeptides of the respective proteins. A number of these hexapeptide submatrices of A is then compared to those of B . The exact number of these hexapeptide submatrices is mainly dependent on the number of C_α atoms of the structures A and B , but is restricted by the authors to limit unnecessary overlap (see "(i) Reduced distance matrix" in [30]). "Similar" pairs of these hexapeptides-hexapeptide "contact patterns" $(i_A, \dots, i_A + 5, j_A, \dots, j_A + 5)$ and $(i_B, \dots, i_B + 5, j_B, \dots, j_B + 5)$ for structures A and B are then added to a "non-exclusive list of pairs", where similarity is scored using

$$S = \sum_{i=1}^L \sum_{j=1}^L \Phi(i, j)$$

where Φ can either be the rigid or the elastic similarity score, $i = (i_A, i_B)$, $j = (j_A, j_B)$ and L is the chunk size. In the case of hexapeptides $L = 6$. Here, i_A, j_A are the i -th and j -th C_α atom indices in structure A and similarly i_B, j_B are the i -th and j -th C_α atom indices in structure B . Measuring the similarity between any of these two 2-tuples can be thought of as measuring how similar the distances between residues i_A, j_A and i_B, j_B are relative to each other. For a detailed descriptions of both the rigid and elastic similarity scores, please refer to

[30]. Now that every pair of contact pattern has a score assigned to it, a Monte-Carlo optimization is performed on the contact pattern pair space to reassemble the hexapeptides into whole proteins. Let S' be the score of accepting a new contact pattern pair and let S be the old score. If $S' \geq S$, a move is always accepted. If $S' < S$ the probability p of accepting a move is given by $p = e^{\beta(S'-S)}$, where β is a parameter that is inversely proportional to the temperature of the system, meaning that for temperatures over absolute zero (which is everywhere in practice) β has a finite upper bound and is greater than zero. This translates to $p \rightarrow 1$ for $\beta \rightarrow 0$. Note that this approach introduces randomness into each pairwise structural alignment. This will get relevant at a later point.

1.3 MSA set representations

In order to be able to perform proper measurements of our MSAs later it is convenient to represent them as sets. We will be giving two definitions. The first one being the set representation for our pairwise structural alignments. The second one will be almost the same, but will focus on pairs of sequences aligned in an MSA. It is helpful to note that these set representations always require exactly two (aligned) sequences, since they represent residue equivalence in the structure alignment case and residue alignment in the case of MSAs.

Definition 3. Pairwise structure alignment set representation

Given two sequences $S_i = (s_{i1}, \dots, s_{ij}, \dots, s_{im_i}) \in \Sigma^{m_i}$ and $S_k = (s_{k1}, \dots, s_{kl}, \dots, s_{km_k}) \in \Sigma^{m_k}$

$$((i, j), (k, l)) \in P_{ref}^{(ik)} : \iff s_{ij} \text{ and } s_{kl} \text{ are structurally equivalent residues} \\ \text{according to some structural alignment algorithm}$$

With $P_{ref}^{(ik)}$ being called the **pairwise structure alignment set representation** for sequences S_i and S_k .

Definition 4. MSA sequence pair set representation

Given two sequences $S_i = (s_{i1}, \dots, s_{ij}, \dots, s_{im_i}) \in \Sigma^{m_i}$ and $S_k = (s_{k1}, \dots, s_{kj}, \dots, s_{km_k}) \in \Sigma^{m_k}$ and the two corresponding aligned sequences in an MSA $S'_i = (s'_{i1}, \dots, s'_{ij}, \dots, s'_{iM}) \in (\Sigma \cup \{-\})^M$ and $S'_k \in (\Sigma \cup \{-\})^M$ and let $g_i(j')$, $g_k(j')$ denote the number of gaps in sequences i and k up until and not including index j' , then

$$((i, j), (k, l)) \in P_t^{(ik)} \\ : \iff \\ s_{ij} \text{ and } s_{kl} \text{ are aligned residues} \\ : \iff \\ \exists j' : j = j' - g_i(j') \text{ and } l = j' - g_k(j') \text{ and } s'_{ij'}, s'_{kj'} \notin \{-\}$$

With $P_t^{(ik)}$ being called the **MSA sequence pair set representation** for sequences S_i and S_k .

We ask the reader to note that the special structure of each element in either of the two sets, which is two 2-tuples inside a 2-tuple, was chosen to avoid unnecessarily complicated notation. At a later point we will be joining these pairwise sets into two unions (one for the structural alignments and one for the MSAs). This union consists of disjoint subsets, their respective sequence pair representations, mainly because the information about what residue pairs stem from which sequence pairs is already encoded within each element. This significantly simplifies notation by not having to worry about aggregating over all columns or rows of the MSAs after the joining of the sets. Other than that the set representations are similar to the description of "homology sets" from the paper "Measuring the distance between multiple sequence alignments" [32].

1.4 Sum-of-pairs-score

The sum-of-pairs-score (SPS) is a popular similarity measure for MSAs. The basic idea is to count how many matches per column for all induced pairwise alignments there are, excluding pairs of the same sequence and pairs of inversed order (e.g. if equivalences between the pair S_i and S_k have been measured, then equivalences between S_k and S_i are ignored) and to sum up all of these column scores for all of the columns of the alignment. A match is a residue pair that is aligned the same as in a reference alignment [33]. To be specific, we define the SPS as follows.

Definition 5. Sum-of-pairs-score (SPS) and comparison strategy

Given an MSA A with alignment rows $S = (S_1, \dots, S_N)$, as well as a **reference MSA** C with alignment rows $R = (R_1, \dots, R_N)$ such that $R = S$, when removing all gaps from each row and let A_{ik} be the MSA sequence pair set representation between alignment rows S_i and S_k and R_{ik} the MSA sequence pair set representation between alignment rows R_i and R_k and $J := \{(i, k) | 1 \leq i < k \leq N\}$, then we call

$$SPS_C(A) := \frac{\left(\bigcup_{(i,k) \in J} A_{ik} \right) \cap \left(\bigcup_{(i,k) \in J} R_{ik} \right)}{\left(\bigcup_{(i,k) \in J} R_{ik} \right)}$$

the **sum-of-pairs score of A according to C** . In general we call any $J \subseteq I \times I$ for a sequence index set $I = \{1, \dots, N\}$ a **comparison strategy**.

Remark. The inspiration for this formulation of the SP-score mainly came from equation 5 in the results section of a paper from B. P. Blackburne and S. Whelan

[32] with the main difference being that we joined several disjoint sets with each other, instead of summing up sizes of the sets across the rows and columns.

There is currently no unanimous definition in literature for what the SPS exactly is. In the introduction we briefly mentioned scoring schemes like BLOSUM and PAM being used in MSA generation to maximize "the SPS". However, this definition of the SPS is different from our definition in that it requires no reference MSA. It works via substitution matrices that "dictate" what residue pair constellation is worth how many "points". Among other things, these scores are decided "based on empirical measurements of amino acid replacement frequencies" [34], which roughly equates to evolutionary closeness. There is no one best scoring matrix and the alignment of different lists of sequences requires the utilization of different scoring matrices depending on how related the input sequences are assumed to be.

1.5 Jaccard distance

The Jaccard distance is a way to measure the difference between sets. It is defined as

$$J(A, B) := 1 - \frac{|A \cap B|}{|A \cup B|}$$

where the similarity measure $\frac{|A \cap B|}{|A \cup B|}$ is called the Jaccard coefficient. $(M, J(A, B))$ is a metric space on the collection of finite sets M , because

$$(a) A = B \implies 1 - \frac{|A|}{|A|} = 1 - 1 = 0$$

$$(b) \text{ obviously } J(A, B) = J(B, A)$$

$$(c) A \neq B \implies \exists a \in A : a \notin B \implies a \in A \cup B \text{ and } a \notin A \cap B \implies |A \cap B| < |A \cup B| \implies \frac{|A \cap B|}{|A \cup B|} < 1 \implies J(A, B) > 0$$

$$(d) J(A, B) + J(B, D) \geq J(A, D) \text{ [35]}$$

The Jaccard distance is particularly useful for comparing complex data, since it is very likely that almost all data has some sort of set representation.

1.6 Classification and accuracy measurement

This subsection is not meant to be a formal introduction to classification theory, but instead focuses on establishing proper terminology to be able to talk about our results. Classification refers to the process of sorting data by one or more shared properties. All elements contained within the same class have been assigned to it by a **classifier** and share some properties that the classifier is able to recognize [36]. In our case we have MSA algorithms aligning multiple sequences to generate a multiple sequence alignment. We will use the previously

defined set representations consisting of two 2-tuples inside a 2-tuple as elements (e.g. $((i, j), (k, l))$ for equivalent residues j, l between sequences S_i and S_k). In a sense each algorithm acts as a (binary) classifier and partitions the output space M , i.e. the set representations of all possible residue pairs between all pairs of sequences, given a fixed set of input sequences, into two sets. Namely the set of **positive outputs** and the set of **negative outputs**. If we now want to measure how "correct" the set of **positive outputs** of any given MSA algorithm is, we can construct a reference set that also partitions the set of all MSAs, again given the same fixed set of input sequences, into positive and negative outputs.

In our case heuristic algorithms generate MSAs, which are measured against our structural alignments of the proteins containing the same sequences as the measured MSAs. What this means is that we get two different classifications into positive and negative ("binary partitions") of the set M . A good performance indicator should reward overlaps, i.e. agreements between regions of the same polarity. Meaning it should reward positive-positive results and negative-negative results, while it should penalize positive-negative results (or the other way around). Whenever residues are aligned in the MSAs that are also structurally equivalent according to some fixed structure aligner, we will be treating those residues as **true positives**. Elements that are aligned, but not equivalent according to the structure aligner are **false positives**. Similarly residues that are not aligned and not structurally equivalent are called **true negatives** and residues that are not aligned, but structurally equivalent are called **false negatives**. These names stem from a method of accuracy measurement called **sensitivity and specificity** [37], which is widely used in natural sciences and medicine, but which we will be borrowing for the description of our sets. In the terminology of sensitivity and specificity, we call our structural reference representation the "**gold standard**".

Before giving the next definition it should be noted that the structure and size of the set M is currently unknown to us. That is why we can not explicitly calculate the absolute values of the specificity for a given alignment. However, we do know the number of false positives for each alignment and this allows the assessment of relative specificity values between different alignments, given that the reference does not change between measurements. This is possible by arguing that a lower false positive rate corresponds to a higher true negative rate, which translates to a higher specificity, if sensitivity values are roughly similar between the measurement of two different alignments.

Definition 6. Sensitivity and specificity of an MSA

Let M be the set of all possible residue pair representations between all pairs of input sequences $S = (S_1, \dots, S_i, \dots, S_n)$ of an MSA t . Let P_t denote any set representation of all pairs of aligned residues between all pairs of sequences in

S of t and let $N_t := M \setminus P_t$ be called the set of negative outputs of t . Similarly let P_{ref} and $N_{ref} := M \setminus P_{ref}$ denote the same for the gold standard "ref". The term

$$\frac{|P_t \cap P_{ref}|}{|P_{ref}|}$$

is then called **sensitivity** of alignment t according to "ref". The term

$$\frac{|N_t \cap N_{ref}|}{|N_{ref}|}$$

is called **specificity** of alignment t according to "ref".

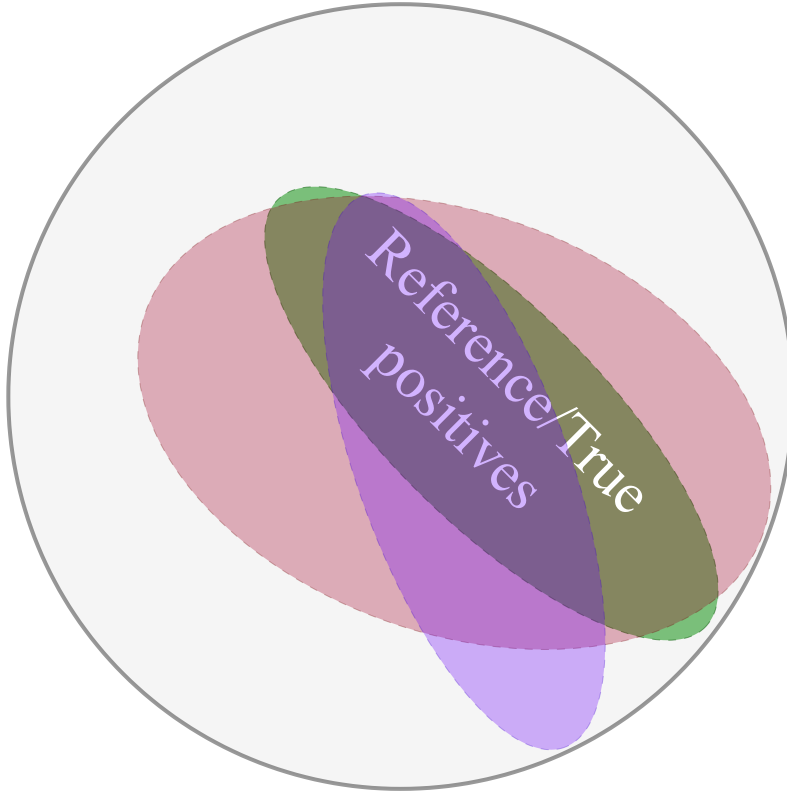


Figure 1: Visualizing example of *definition 6*. The gray ellipse represents the confines of our unknown set M . Green represents P_{ref} , i.e. the set of equivalent residue pairs in the reference. Red and purple are two different sets of aligned residues, let them be called P_{red} and P_{purple} , as returned by two different MSA algorithms. From this example we can see that the red aligner would have higher sensitivity than the purple one, since it incorporates almost all of the green set. Whereas the purple aligner would have higher specificity, since it incorporates less of the non-green (more of the gray) region than the red aligner. Thus when looking for a suitable scoring system, the problem can be simplified to rewarding the overlap of different sets of aligned residues all while penalizing everything not contained in the set intersections.

More generally sensitivity is the ability of a given test to correctly identify true positives, whereas specificity is the ability to correctly identify true negatives. A test that always returns true maximizes sensitivity, and a test that always returns false maximizes specificity. Thus, it is important to look at both sensitivity and specificity in tandem. It is a reasonable assumption that the set of true negatives vastly exceeds the set of false positives in size, meaning that there are far more ways to "wrongly" align residues than just the one specific wrong alignment that the algorithm chose. Thus, since we do not know the exact set of true negatives without further investigation, we can write the specificity as ratio of the number of true negatives $|X|$ to the sum of the numbers of true negatives and false positives like

$$\frac{|X|}{|X| + |P_t \setminus (P_t \cap P_{ref})|}$$

Using our assumption, which is $|X| \gg |P_t \setminus (P_t \cap P_{ref})|$, we can estimate the specificity to be

$$\frac{|X|}{|X| + |P_t \setminus (P_t \cap P_{ref})|} \approx \frac{|X|}{|X|} = 1$$

However, there are many more things to consider before we would draw the conclusion that specificity does not matter. For one, not all possible falsely aligned residues are accessible to the MSA algorithm at any given time. For example, if we were to align the first residue of a first sequence with the last residue of a second sequence, there is no way how any other residues could possibly be aligned between those two sequences. It is not clear to us at the moment what consequence this bears for specificity in particular or for the structure of set M in general.

In the following we will be judging sensitivity and specificity on several different levels. Firstly on the level of heuristic MSA algorithms, meaning that we assume that our structure alignments are the "gold standard", as it is called in medical diagnostics and we apply the definitions of sensitivity and specificity accordingly. But on a higher level we also need to be able to judge sensitivity and specificity of our reference against the optimal MSA for each given set of inputs. This is a more difficult task, because we are in dire need of a reference for our reference, inevitably resulting in a potentially endless chain of better and better approximations of what we call reality. Now, while this is exactly what we do in science in a broader sense, it is obviously not a viable thing to do here. Hence, instead of constructing another ground truth for our ground truth, we will be trying to judge sensitivity and specificity in relative terms. This means that if we modify our reference sets in any way (which might be necessary for the application of theorems and the like), we will be trying to judge, given the limited information we have about our reference set, if sensitivity and specificity

remain the same or get better or worse with these modifications, **relative** to the sets without modifications. And then finally, we could also ask the question of how much the problem of MSA, given any or one specific similarity measure, represents the true evolutionary process in the first place. Although latter will not be of any major concern to us in this work.

2 Methods

Like mentioned in the abstract we developed a software to perform and test the approach of measuring MSA quality via pairwise structural alignments. We proceed to give a rundown of the general procedure of score calculation and provide mathematical justifications for each step where possible.

2.1 Set up and protein database

The "*Protein Data Bank*" (PDB) [38] is a database of more than 200 000 protein structures mostly obtained via NMR-spectrometry or X-ray diffraction. Our software keeps a local copy of it that can be kept up-to-date via the option

```
python3 -m PSTAR-MSA --syncdb --diamondfile <path/*.dmnd>
--locpdbdb <path> --outputdir <path>
```

This will also create/update a FASTA file containing all sequences from the local PDB files. As for right now the overall process of generating the FASTA file takes a long time. Mostly due to the parsing of the more than 200 000 PDB files. We cannot certainly rule out that it is bad programming on our side, but for now we suspect that the *PDBParser*, i.e. its called submodules are slowing down the FASTA file generation significantly. With our current approach it takes several hours to write all of the data into the file. At first the *SeqResIterator* from the *Biopython* [39] package was used to parse PDB data, but this parser processes PDB files differently from how DALI processes PDB files, which ultimately led to a mismatch between parsed structural data that was used to perform a local protein data base search and the protein data that was being used by DALI to generate the structural alignments. Hence, we decided to switch to the *AtomIterator*, also found in the *Biopython* package, which minimized or even resolved the mismatch issue entirely. In possible future updates we might switch from storing PDB files to the more up-to-date file format of mmCIF, which might also fix slow parsing speeds. We provide a relatively up-to-date FASTA and custom index file used for local database updates on our GitHub page to accelerate the setup process. Once both files are in place, it is only updated according to new entries in the PDB, which should not take too long. However, PSTAR-MSA

also will generate a fresh FASTA file, if none is present. Manual edits of the FASTA file and subsequent data base generation will still have to be performed occasionally, if PDB removes entries for whatever reason. This is because, while we implemented a function to automatically append the FASTA file according to new PDB entries, we have not yet implemented a function to remove outdated entries from it. Removal of PDB entries is a rare occasion, but it happens from time to time.

After setting up the local PDB copy and corresponding FASTA file, we use *DIAMOND* [40] to set up a proper database to find the corresponding protein structures to our sample of query sequences. A full list of dependencies/prerequisites can be found on our GitHub page, as well as a description of the functionalities and outputs.

2.2 Data availability and data bias

To ensure that there is sufficient reliable data available to perform benchmarks on, we analyzed the HomFam reference data set [41]. More specifically, we carried out a search for every sequence in *HomFam* including homologues against our DIAMOND database of the whole PDB and evaluated the results to decide how we should best define our sequence filter to find a good balance between data availability and result confidence. It is important to note that the *HomFam* reference data set contained several duplicate sequences per protein family, as others have also pointed out before [42]. We decided to exclude these duplicate sequences for our purposes and selected one alignment row of these sequences at random as a representative for each set of duplicates.

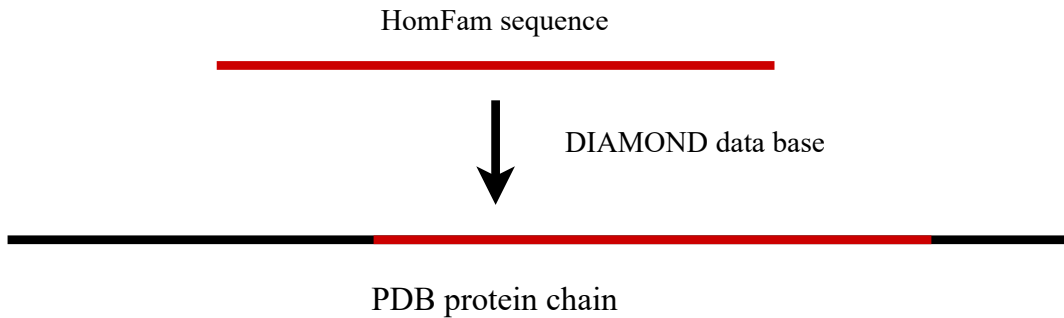


Figure 2: Searching for a protein chain in PDB using DIAMOND given a query sequence (red)

It should be noted that filtering results in any way could lead to bias in the data and threaten general applicability of our approach of benchmarking. Generalizing reference data sets was one of the main goals in the first place. Experimentally measuring protein structures via most reliable methods is not only

time-consuming, but also expensive. Naturally this can lead to certain proteins or protein families being measured more frequently than others, be it for medical, economic, technical or other reasons. If we now only choose those sequences, for which we can be sure that they stem from a protein that has actually been measured, we might accidentally discriminate against those structural motifs that are known to be more common in other protein families than those that are most prominently investigated. Which in turn might lead to those algorithms scoring higher, which are better at aligning these specific protein families, but might not be more accurate in general. Either way, we chose to ignore the looming threat of yet another echo chamber for now and trust that our filter has a negligible effect on bias. Therefore we only included exact query matches in our calculation, which we will be defining as follows:

Definition 7. Exact query match

A subject sequence $M_i = (s_1, \dots, s_{m_s}) \in \Sigma^{m_s}$ is called an **exact query match** for query sequence number i , if there is a query sequence $Q_i = (q_{i1}, \dots, q_{im_i}) \in \Sigma^{m_i}$ with $m_i \leq m_s$ that is a substring of M_i . In that case we write

$$M_i := (s_1, \dots, s_{sms_i-1}, q_{i1}, \dots, q_{im_i}, s_{sms_i+m_i}, \dots, s_{m_s})$$

We call sms_i the **subject match start** and $I_i := \{sms_i, sms_i + 1, \dots, sms_i + m_i - 1\}$ is called the **subject-embedded query index set**.

Remark. To put into simple words: We include only those sequences in our calculation that are a complete and continuous substring of a sequence in our database. And I_i is just the index **range** set starting from the index sms_i , where the query sequence with index i first matches the subject, and ending at and including index $sms_i + m_i - 1$. If any character of a query sequence, including the last, is not matched by the subject, this is **not** considered an exact query match.

This way we guarantee to the best of our abilities that only those sequences are used where the original protein has experimental structure data. The longer the query sequence in the corresponding exact query match, the higher the confidence that the structure data is taken from the exact protein that we are trying to measure. The problem of bias could later be solved by also including high-confidence protein structure predictions like AlphaFold. This would also increase data availability.

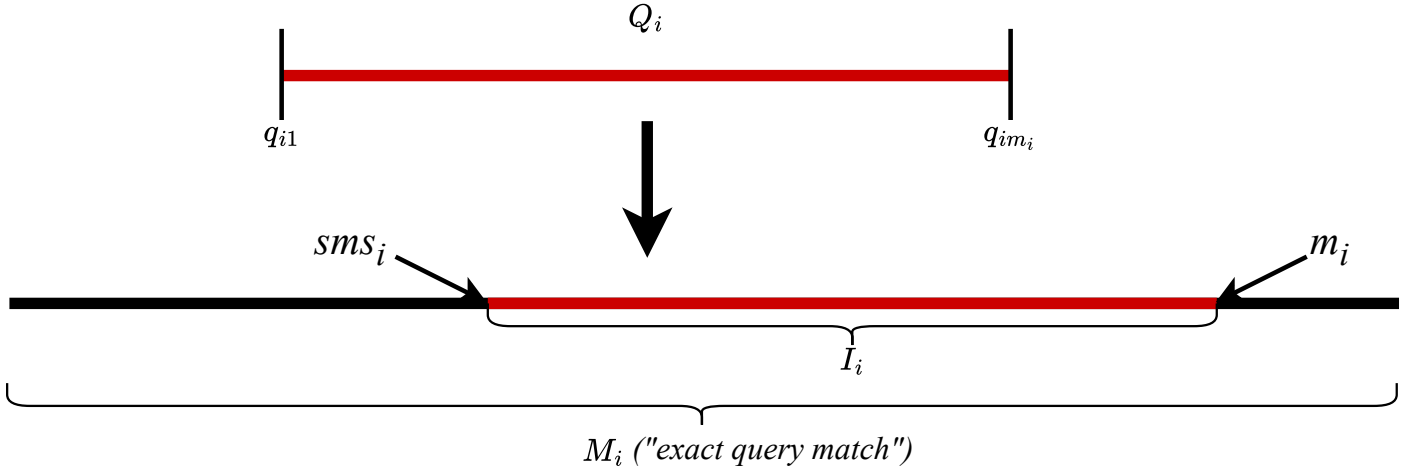


Figure 3: Illustration of *definition 7* and its remarks

2.3 General programmatic procedure and input data

As input data we used the output of several different MSA algorithms, which were tasked to create multiple sequence alignments for all sequence families in HomFam. Those algorithms are listed and described in *Subsection 1.1.1*. Additionally, the HOMSTRAD (HomFamRef) reference alignments for the same sequence families will also be used as input.

After placing the outputs of each MSA algorithm for one family in the same directory, then in a first step the alignments are cleaned of their gaps and these lists of gapless sequences are then passed on to DIAMOND as FASTA files, which returns query responses as TSV files, containing information about each query file's database matches. (e.g. sms_s, m_q , PDB-IDs, number of gaps, e-value and so forth). Each query file is associated with a sequence family and an MSA algorithm. Given that we have 94 sequence families in HomFam and 7 MSA algorithms and HomFamRef, this amounts to a total of 752 query files. However, we also included an older version of learnMSA, as well as something we call baseline alignment.

The baseline alignment is generated by scanning all input alignments for the shortest length alignment and then reading all common sequences between the alignments and subsequently inserting gaps at random positions in each sequence, until the length of the shortest alignment is met for each sequence. This alignment will later serve as a reference of how much of the score can at maximum be attributed to randomness or the differences in alignment lengths. If less gaps have to be inserted overall, because the alignment length is shorter, it is more likely that a sensitivity measure scores these alignments higher than longer alignments. This is simply due to the fact that a sensitivity measure does not penalize wrongly

aligned sequences. As a rule of thumb, we can expect that scores associated with a higher scoring baseline alignment are less accurate overall.

We then proceeded to run a database query via

```
<path>/DIAMOND blastp\
  --db <db_file_path> --query <FASTA_file_path> -o <output_path>\
  --log --header simple --outfmt 6 qseqid sseqid pident length\
  mismatch gapopen qstart qend sstart send evalue bitscore qlen\
  --id 99.9
```

From the returned TSV files of the database queries we can then proceed to gather all of the exact query matches. It can happen that a sequence matches several different protein chains in our database. This is either due to duplicate entries with different names within the protein data bank itself or because a particularly short sequence was queued, which increases the likelihood of matching to two different chains of two different proteins. The latter can probably safely be treated as a rare occurrence. Either way, it will later become apparent, why it is important to select the same exact query match, given the same input sequence. That is why we developed a deterministic selection process to have us return the same protein, given that DIAMOND always returns the same selection of results for a given query sequence. If we are returned more than one potential candidate for an exact query match, we first try to select the one with the lowest *e-value*. If the *e-values* are identical, we select the one with the highest search score. After that it is pretty much irrelevant which match we prefer over the other and we choose the one with the lowest *sms_s*. And if those should be the same as well, we choose the protein chain based on lexicographical ordering of the PDB-IDs. Now that we have a one-to-one correspondence table between aligned sequences and PDB-IDs, we add all of these sequences to sets and intersect all of them. The intersection will contain all common sequences between all of the input alignments that also have exact query matches in our DIAMOND database. We assign common sequence indices to all of them to be able to identify them later.

These common sequence indices can be described as our set $I = \{1, \dots, N\}$ for a total of N common sequences. We proceed to generate a **comparison strategy** $J \subseteq I \times I$, which is planned to be customizable at a later point. For now we chose J to be the same as in the definition of the SPS. Based on the comparison strategy (or any subset of J) and for all $(i, k) \in J$ we calculate the structural alignments between exact query matches M_i and M_k via DALI that correspond to common sequences C_i and C_k . For this we first imported all the necessary PDB files by first importing them into DALI DAT format via

```
<path>/import.pl --pdbfile <PDB_file_path> --pdbid <PDB_id>\
  --dat <DAT_output_path> --clean\
  1 > <stdout_log_file> 2 > <stderr_log_file>
```

and then generating the actual alignments for each pair of sequence indices in our sample subset of the comparison strategy via

```
<path>/dali.pl --cd1 <query_chain_id> --cd2 <subject_chain_id> --dat1\  
  <path_to_query_dat_file> --dat2 <path_to_subject_dat_file>\  
  --title <job_name> --clean\  
  --outfmt summary,alignments,equivalences,transrot
```

This process is the bottleneck, meaning that it takes the longest out of all steps for a large amount of alignments. DALI technically offers an *OpenMPI* [43] option for multi-threading and we also implemented our own multi-threading version of it, which generates single alignments in parallel. But both of these options are still not fully implemented. We also implemented an option to continue each alignment generation process, should it fail part-way through. This is useful when calculating large numbers of structural alignments.

After alignment generation we proceed to parse the structural alignment data and calculate their respective set representations with the following definition of structurally equivalent residues according to DALI.

Definition 8. Structurally equivalent residues according to DALI

Given two sequences $S_i = (s_{i1}, \dots, s_{ij}, \dots, s_{im_i}) \in \Sigma^{m_i}$ and $S_k = (s_{k1}, \dots, s_{kj}, \dots, s_{km_k}) \in \Sigma^{m_k}$ and the two corresponding structurally aligned sequences $S'_i = (s'_{i1}, \dots, s'_{ij'}, \dots, s'_{iM}) \in (\Sigma \cup \{-\})^M$ and $S'_k \in (\Sigma \cup \{-\})^M$ and let $g_i(j')$, $g_k(j')$ denote the number of gaps in sequences i and k up until and not including index j' .

s_{ij} and s_{kl} are structurally equivalent residues according to DALI

$:\iff$

$\exists j' : j = j' - g_i(j')$ and $l = j' - g_k(j')$ and $s'_{ij'}, s'_{kj'} \notin \{-\}$ and $s'_{ij'}, s'_{kj'}$ are upper case

```

No 1: Query=5GU4B Sbjct=1QCGB Z-score=33.4

DSSP  LLEEEEEELLLLHHHHHHHHHHHHHHHLLLLLEELLEELLLLLLLLhhHLEEEEEEEELL
Query  YPIINFTTAGATVQSYTNFIRAVRGRLTTGADVRHEIPVLPNRVGLpinQRFILVELSNH 60
ident  | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
Sbjct  VNTIIYNVGSTTISKYATFLNDLRNEAKDPSLKCYGIPMLPNTNTN---PKYVLVELQGS 57
DSSP  LLEEEEEELLLLHHHHHHHHHHHHHHHLLLLLEELLEELLLLLLLL---LLEEEEEEEELL

DSSP  LLEEEEEEEELLLLLEEEEEEL-----LEEEELLLLHhhHHHLL---LLLLL---LEE
Query  AELSVTLALDVTNAYVVGyrAG-----NSAYFFHPDNQedaEAITH---LFTDV---QNR 109
ident  | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
Sbjct  NKKTITLMRLRRNNLYVMGYSDPfetnkCRYHIFNDISG---TERQDvetTLCPNansRVS 114
DSSP  LLEEEEEEEELLLLLEEEEEEEellleEEEEELLLLLL---HHHHHhhHHHLLl111EEE

DSSP  FELLLLLHHHHHHHHHLL-LHHHLLLHHHHHHHHHHHHHHHLLLLLHHHHHHHHHHHHH
Query  YTFAGGNYDRLEQLAGN-LRENIELGNGPLEEAISSALYYysTGGTQLPTLARSFIICIQ 168
ident  | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
Sbjct  KNINFDSRYPTLESKAGVksRSQVQLGIQILDNSNIGKISG--VMSFTEKTEAEFLVIAIQ 172
DSSP  LLLLLLLLHHHHHHHHHLLlLHHHLLLHHHHHHHHHHHHHLL--LLLLLHHHHHHHHHHHHH

DSSP  HHHHHHHLHHHHHHHHHHHHHlLLLLLLLHHHHHHHHHHHHHHHHHLLLLLEEEEEEEEEE
Query  MISEAARFQYIEGEMRTRicyNRRSAPDPSVITLENSWGRNSTAIQESNQAFASPIQLQ 228
ident  | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
Sbjct  MVSEAARFKYIENQVKTNF--NRAFNPNPVKVLNLQETWGKISTAIHDAKNGVLPKPLELV 230
DSSP  HHHHHHHLHHHHHHHHHHHLL--LLLLLLLHHHHHHHHHHHHHHHHHLLLLLEEEEEEEEEE

DSSP  LLLLLLEEEEEELHHHLLLLLLLLLLLLL-----
Query  RRNGSKFSVDVSIPIIAlMVYRCA----- 255
ident  | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
Sbjct  DASGAKWIVLRVDEIKPDVALLNYVGGscqtt 262
DSSP  LLLLLLEEEEEEHHHHHHHHLLLLLLLLLLl1111

```

Figure 4: Example of DALI output between protein chains 5GU4B and 1QCGB. Red rectangles surround structurally equivalent subsequences. Let S_q denote the query sequence and S_s the subject sequence. We can see that the last residues, namely $s'_{q(60)}$ and $s'_{s(60)}$ in the first row are structurally equivalent, because neither **H** nor **S** are gap symbols and both are upper case. Then with $j' = 60$ we have that residue $s_{q(60-0)}$ and $s_{s(60-3)}$ are equivalent, because $g_q(60) = 0$ and $g_s(60) = 3$.

The set representations are then also calculated for the aligned MSA sequences (see *definition 4*). For each pair of sequences C_i, C_k used to generate the latter set we then add sms_i, sms_k to the column indices (e.g. $((i, j), (k, l)) \rightarrow ((i, j + sms_i), (k, l + sms_k))$ for all columns j in C_i and all columns l in C_k) to make the column indexing match the position of the sequence snippet within their respective exact query match proteins M_i and M_k (look ahead at *definition 9*). We also modify our reference set to cut off those parts of the structurally aligned sequences, where there is no overlap between the original MSA sequence snip-

pets in the pairwise structural alignment (look ahead at *definition 10*). After all of these modification we are left with families of disjoint sets, one family for each of the MSA set representations and the structure residue equivalence set representations, which we then proceed to join into larger sets to simplify notation (look ahead at *definition 11*). This way we acquire one set for each MSA algorithm as well as a set for HomFamRef and the baseline. We save the sets in JSON files. This way it is still possible to use them at a later point without needing to go through the alignment generation process again. We also generate a detailed overview of the disjoint parts (which represent sequence pair equivalences/aligned residues) of the union sets to be able to see which pairwise alignment contributed how much to the overall score and to calculate a distribution, should one desire to do so.

2.4 Scoring

If we want to be able to **fairly** compare $P_{ref}^{(ik)}$ and $P_t^{(ik)}$ and assign a score to alignment t , we need to make some modifications to $P_t^{(ik)}$, because we assume that in general the shorter query sequence from the MSA is embedded as a single, continuous chunk somewhere in the corresponding subject protein sequence. Therefore we will be constructing a modified MSA sequence pair set $P_t^{(ik)'}$ to account for this fact.

Definition 9. *Subject-embedded MSA sequence pair set*

Given an MSA sequence pair set $P_t^{(ik)}$, then the set $P_t^{(ik)'} := \{((i, j'), (k, l'))\}$ is called the **subject-embedded MSA sequence pair set**, with $j' \in I_i$ and $l' \in I_k$. We construct it via

$$P_t^{(ik)'} := \{((i, j + sms_i - 1), (k, l + sms_k - 1)) \\ \mid \forall j \in \{1, \dots, m_i\} \text{ and } \forall l \in \{1, \dots, m_k\}\}$$

Remark. We can see that this construction is consistent with Def.6 when looking at j' (or l' respectively). Should the query sequence start with $j = 1$ and end with m_i , then $\min(j') = 1 + sms_i - 1 = sms_i = \min(I_i)$ and $\max(j') = m_i + sms_i - 1 = \max(I_i)$.

Next we will be restricting our pairwise structure alignment set representation to only include those residue equivalences, where the original sequence snippets from the MSA overlap in the exact query match protein. We start by defining the exact query match restriction.

Definition 10. *Exact query match restriction*

Given a pairwise structure alignment set representation $P_{ref}^{(ik)}$ of two structurally

aligned sequences S_i and S_k , which have exact query matches M_i and M_k , as well as their corresponding subject-embedded query index sets I_i and I_k , then

$$P_{ref}^{(ik)'} := P_{ref}^{(ik)} \setminus \{(i, j), (k, l) \mid i, k \in \{1, \dots, N\} \text{ and } j, l \notin I_i \cap I_k\}$$

is called the **exact query match restriction**.

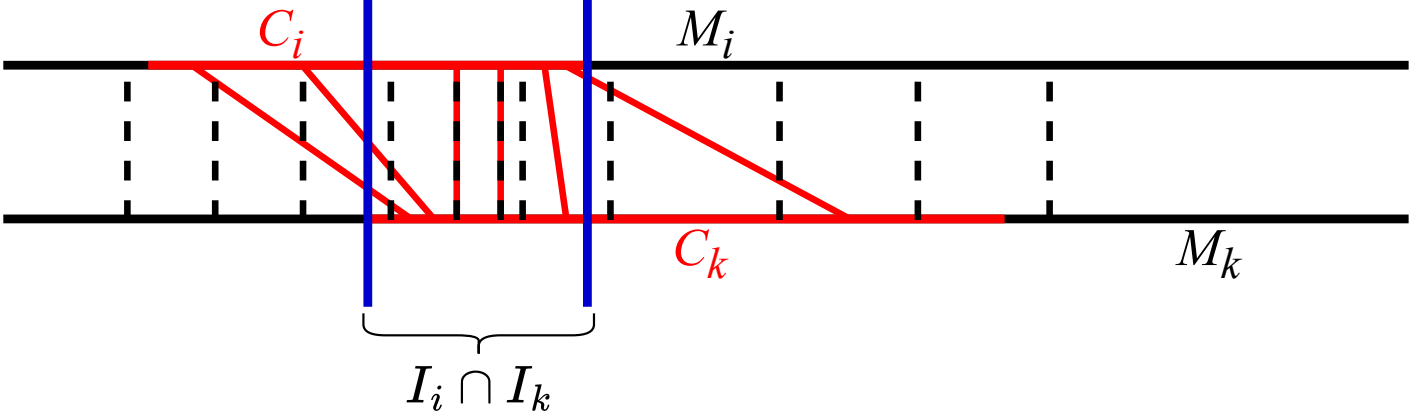


Figure 5: Illustration of *definition 10*. C_i and C_k are MSA sequences and M_i and M_k their respective exact query matches. Dotted black lines are structurally equivalent residues and red lines represent residues that are aligned in the original MSA. In this simple case, a sensitivity measure would give a value of 0.5 for the pair (C_i, C_k) , since 2 red lines match with 2 of the 4 black lines in the blue interval, while the third red line is ignored by this measure.

By removing all the flanking sequences not contained within the original query we don't remove true positives from the score, since the MSA algorithm cannot align what is not contained within the original sequences. Prior to this step our score would falsely evaluate all these elements as being false negatives, when in reality they should be treated as unclassifiable by the MSA algorithm. Next we need to choose an alignment strategy. We defined the reference set $P_{ref}^{(ik)}$ for a pair of sequences S_i and S_k . Which is fine, if we were trying to measure pairwise sequence alignment quality. Since MSA sequence residue equivalences are transitive (i.e. if $a \sim b \wedge b \sim c \implies a \sim c$ for aligned sequence residues a, b, c), but pairwise structurally aligned residue equivalences are generally not and we artificially set a reference for how two sequences in the MSA should be aligned that is potentially unachievable or not even part of the optimal MSA solution set (independence of pairwise structural alignments vs. transitivity of residue alignment in MSA), we might falsely classify a residue alignment as false that might actually be optimal within the problem constraints of MSA. This leads to a decrease of sensitivity and specificity of our reference, which just means

that it is not in our reference set, although it really should be and additionally that there are elements in our reference that can only be achieved by chance or not at all. This is a fundamental problem with this approach that can not be accounted for at the moment. In the future we could calculate a confidence score that is based on a "transitivity deviation measure" of sorts among other things. To sum it up, we increased the specificity of our approach by performing the exact query match restriction, which lowers false positive classification of residues of a pairwise sequence alignment that the MSA can not achieve. We then proceeded to elaborate why our approach still does not have perfect specificity or sensitivity, since there are regions in the true MSA solution residue equivalence set, which our approach **can not** recognize as optimal, because pairwise structure alignment in itself does not "understand" the constraints of aligning multiple sequences.

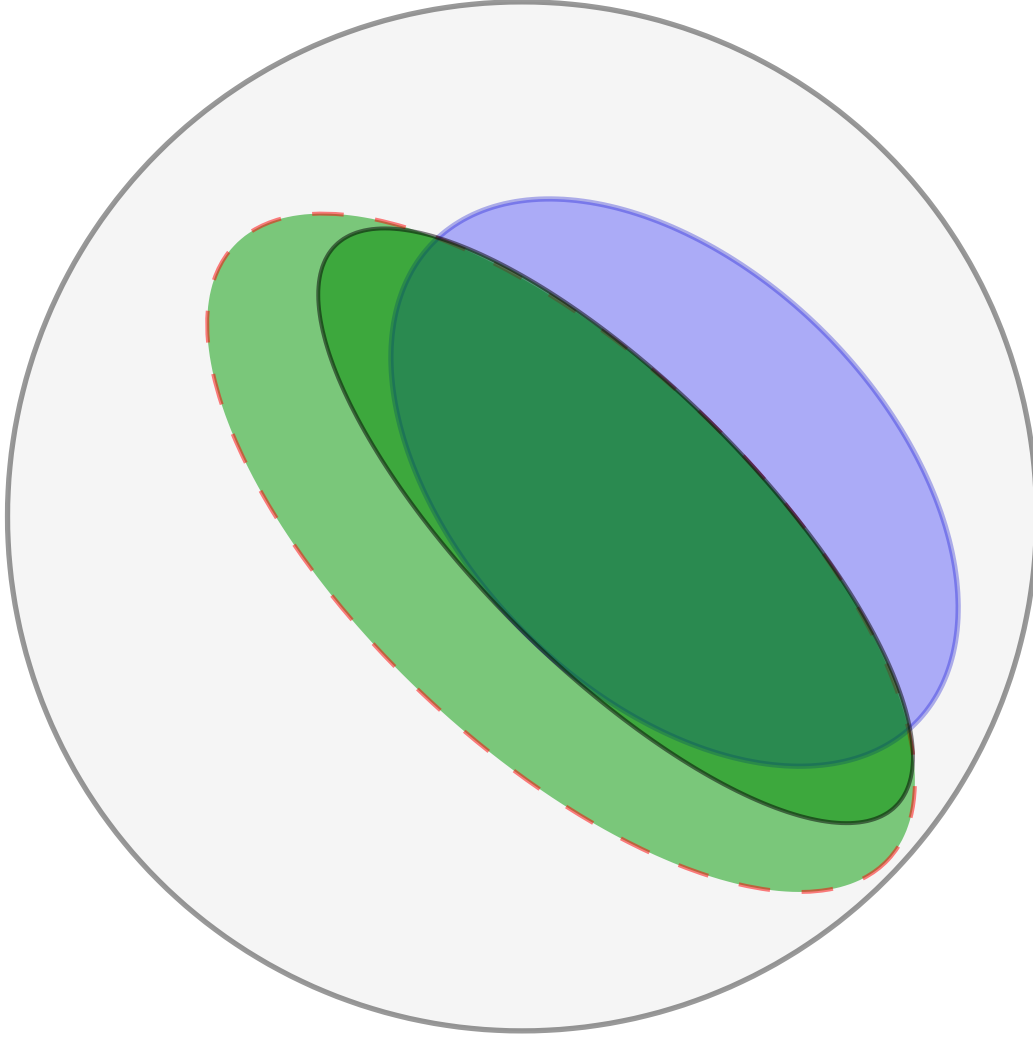


Figure 6: The blue ellipse represents the "optimal" MSA solution, which acts as "gold standard" on a higher level than our reference, latter is represented by the green ellipses. The light green region that is confined by the red, dotted line represent $P_{ref}^{(ik)}$, i.e. our reference set before applying *definition 10*. Whereas $P_{ref}^{(ik)'}$ is represented by the dark green region, i.e. after applying the query match restriction. The blue regions that do not overlap with the green regions are regions that are unclassifiable by the structure alignments, e.g. for reasons of transitivity. Note that visual overlaps between blue and light green are not intended, since we already established that the light green elements are unclassifiable.

Definition 11. : PSTAR-union

Given a subject-embedded MSA sequence pair set $P_t^{(ik)'}$, an exact query match restriction $P_{ref}^{(ik)'}$ and $I = \{1, \dots, N\}$ for N test sequence indices, as well as a comparison strategy $J \subseteq I \times I$. The function

$$u_J : P_*^{(ik)'} \mapsto \bigcup_{(i,k) \in J} P_*^{(ik)'} = P_*^u$$

is called the **PSTAR-union** for comparison strategy J with respect to $* \in \{ref, t\}$.

We chose $J = \{(i, k) | 1 \leq i < k \leq N\}$ as a comparison strategy, because we make the simplifying assumption that DALI generates identical alignments for the same sequence inputs no matter the order of the inputs. This is not an exact rule, because it is not even guaranteed that DALI generates the exact same alignment for the same input, since part of its algorithm uses a Monte-Carlo approach.

Definition 12. : PSTAR-score

Given a list of N alignment rows $A = (A_1, \dots, A_N)$ with unique underlying sequences $C = \{C_i\}_{i \in I}$ and $I \subseteq \{1, \dots, N\}$, such that for each $i \in I$ there exists an exact query match for C_i , then

$$PSS_C(A) := \frac{|P_A^u \cap P_{ref,C}^u|}{|P_{ref,C}^u|}$$

is called the **PSTAR-score**.

This score simply measures the overlap (the agreement) between the union of all MSA sequence pair set representations of a given MSA and our structural equivalence set, while ignoring the disagreements for now. We actively decided against using the Jaccard index for this, because the SPS analogue, which we simply called the PSTAR-score is a common way to measure alignment quality and allows us to more directly compare our results to others. On the other hand, we might score shorter alignments higher than longer alignments, because it simply increases the likelihood of any given pair of residues to be aligned, when there are less gaps to be inserted into each MSA input sequence overall. The PSTAR-score closely resembles the sum-of-pairs score, and is a sensitivity measure. This can also be seen by writing down the sensitivity as the fraction of true positives $P_{ref,C}^u \cap P_t^u$ of all actual positives P_{ref}^u . However, we would also like to be able to directly measure the distance between MSA sequence representation sets using P_{ref}^u as reference. Defining a proper metric for this turns out to be quite a challenge. The problem with the PSTAR-score itself is that comparing results among different MSA algorithms is not directly reliable. For example simply consider the difference

$$|PSS_C(A) - PSS_C(B)|$$

For ease of notation we will write $A := P_a^u$, $B := P_b^u$ and $C := P_{ref,c}^u$. Then suppose we have $A = \{1, 2, 3, 4\}$, $B = \{5, 6, 7, 8\}$ and $C = \{3, 4, 5, 6\}$. We then get

$$PSS_C(A, B) = \left| \frac{|\{3, 4\}|}{4} - \frac{|\{5, 6\}|}{4} \right| = 0$$

Thus, $A \neq B \not\Rightarrow PSS_C(A, B) > 0$, which is violating positivity, which is one of the required properties of a metric on the collection of all sets. There might still be the possibility that our set representations with all their modifications have a certain structure that disallows configurations like the ones used in this counter-example. However, in general the lack of positivity translates to the measured sets appearing to be similar, although they are not. But we still have $PSS_C(A, B) > 0 \implies |A \cap C| \neq |B \cap C|$ and $PSS_C(A, B) = 0 \implies |A \cap C| = |B \cap C|$, which is what we are actually interested in primarily. But algorithms that just align more residues with each other in the crucial reference interval will score higher, up to a point where they could simply align all residues in the reference interval. Hence, maximizing the PSTAR-score in a specific MSA problem just means correctly guessing the reference interval and then aligning every residue in it, regardless of biological meaning. This is especially relevant with the introduction of artificial intelligence MSA algorithms like learnMSA. That is why we would also like to measure the specificity, meaning that we would like to penalize wrongly aligned residues within the reference interval. The Jaccard index is a very good choice for that in the author's opinion. It measures the agreement in the numerator and the disagreement in the denominator. However, it suffers from the same problems when wanting to compare two different scores via looking at simple distances between them (see *Corollary 1, (3)*). That being said, it does not pose a real problem when trying to measure alignment quality, since we are only concerned with rewarding true positives and penalizing false positives as well as false negatives. Let us take a step back and take a look at what the Jaccard index would measure in our context. In the numerator we have $|A \cap C|$, which is equivalent to the number of true positives. In the denominator we have $|A \cup C| = |A \setminus C| + |C \setminus A| + |A \cap C|$, which is nothing else than the sum of all true positives $A \cap C$, all false positives $A \setminus C$ and all false negatives $C \setminus A$. With the exception of the summand $A \setminus C$ this is nothing else than the sensitivity of our MSA algorithm with the difference that the score can be lowered further by false positives. If we observe similar values for the Jaccard index, we can conclude for the distances between them that, since C is constant between each measurement, the ratios of (true positives)/(false positives) must be the same. This then means nothing else than if one of the sets A or B is bigger, it must "compensate" for that by having a larger overlap with C to achieve the same score as the other. Thus, it is not an effective strategy anymore to just align everything in the crucial interval and "save gaps" for outside the interval.

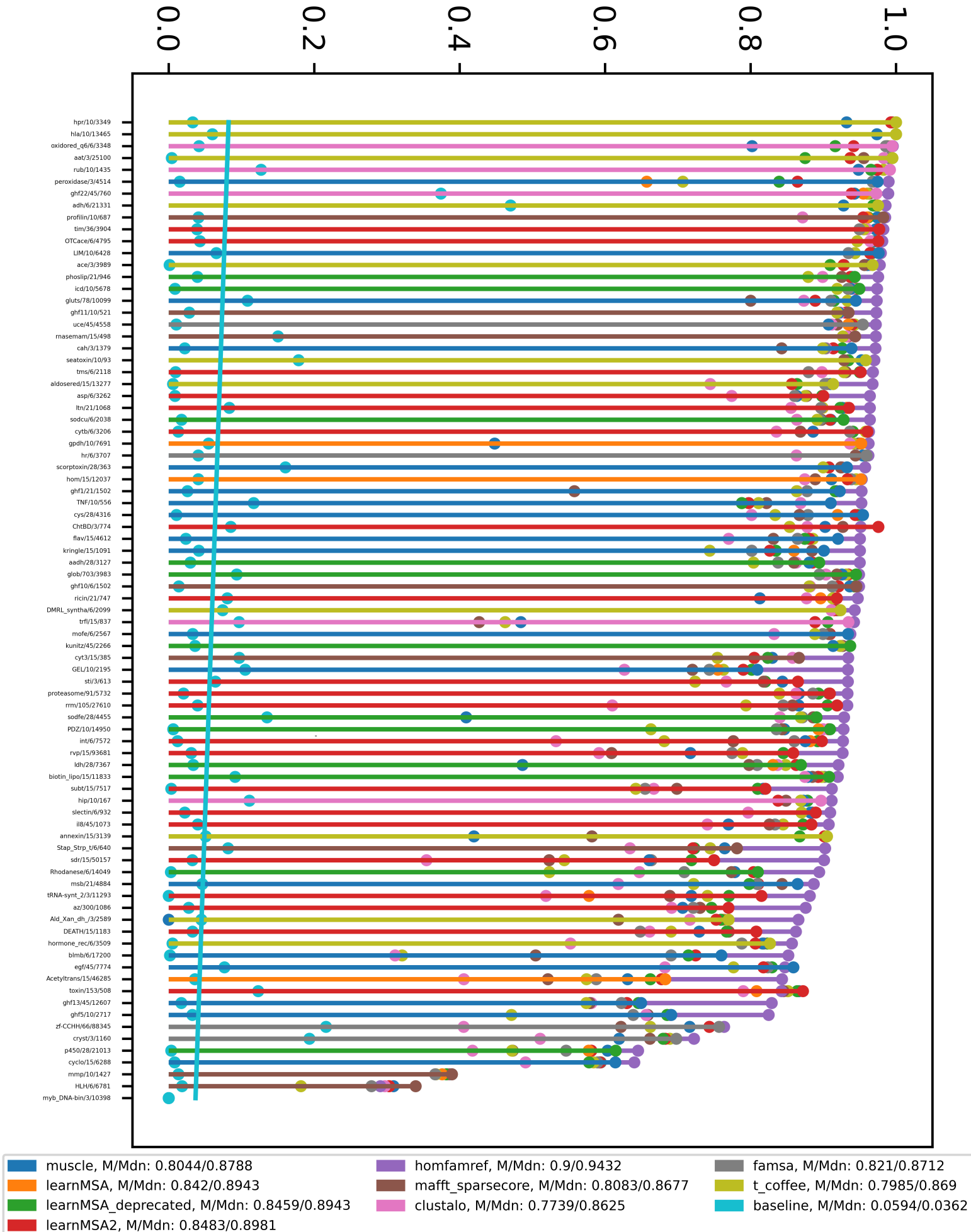
3 Results and discussion

3.1 Scoring MSA algorithms with HomFamRef

In a first step we measured the PSTAR-scores of each MSA algorithm listed in *Subsection 1.1.1* for each HomFam protein family together with the corresponding HOMSTRAD reference for each family. Since the HOMSTRAD reference does not contain homologue sequences, data availability was limited by the number of distinct sequences in each of the family's reference alignments. Our approach makes it necessary to compare only sequences that are common in all compared input alignments. This means that technically we cannot even compare results between different families of proteins, because those refer to different common sequence sets. However, despite not having certainty regarding the **exact** uncertainties that comparing different families with each other brings with it, it will be a rough indicator on how well the different HomFam families can be measured with the PSTAR-score.

Fig.7 below shows an overview of all acquired results. Each entry on the stem root axis represents a HomFam protein family vs. a PSTAR-score for each aligner on the other axis. The stem root axis does not only contain the name of the protein family, but also the theoretical number of structural alignments used as reference after the first slash, as well as the number of aligned sequences (total number of aligned sequences, homologues + HOMSTRAD sequences) after the second slash. We say "theoretical number" of structural alignments, because it should refer to data availability and not to the capabilities of one specific structural alignment algorithm. If n is the number of HOMSTRAD sequences that have exact query matches and are common in all MSA outputs, then $\frac{n^2-n}{2}$ is the number that can be read after the first slash. This is due to our choice of comparison strategy. There are instances, where DALI can not or will not generate a structure alignment. This can happen due to several different reasons. One example would be PDB entry *IMMC*, which as of the time of writing this, simply crashes DALI (even so the web application) during the structure import process. However, these instances are comparatively rare and do not skew the results a lot, and it is often obvious from the obtained data when something is wrong (see the very last entry in *Fig.7* for example). The entries have been sorted in descending order by the score of the HomFamRef alignments (purple). Unfortunately the entry names on the stem root axis can only be read in the PDF document, because they need to be very small to fit them all onto the image in an A4 readable size, but they can be found on our GitHub page and are also only of minor concern for the general data evaluation.

Figure 7: HomFam protein family MSAs and PSTAR-scores of chosen aligners; baseline regression: $y = -0.0005543x + 0.08212$, $R^2 = 0.03177$



Continuing the evaluation of *Fig.7* we can see that almost always the HomFam reference is the highest scoring alignment. Which is good, because it is actively being used as a reference for evaluating MSA quality. However, consider the example of protein family "adh", which is the 8th alignment starting from the top. Note that it has an exceptionally high scoring baseline alignment. Recall from our description of the baseline alignments that we used the shortest alignment length per family as the length of our baseline alignment. For family "adh" HomFamRef has a length of 375. The lengths of the other alignments are 3452 (T-COFFEE), 936 (learnMSA_deprecated), 1020 (CLUSTALO), 1475 (FAMSA), 2470 (MAFFT-sparsecore), 2459 (MUSCLE), 840 (learnMSA2) and 915 (learnMSA). Now also recall that we simply inserted gaps at random positions in the baseline alignments and it scores around 0.48. That is not a bad score for **RandAl**, which is what we could call our aligner, but we will not. Since the PSTAR-score only measures positives and ignores the inclusion of false positives, it is prone to misjudging alignment quality as too high and can give high scores, despite the alignments perhaps having no biological relevance. The same is also true for the SP-score in general. The MSA algorithms always have the "choice" to align uncertain regions with gaps, so that they are not represented in our index sets. Thus we feel that penalizing wrongly aligned residues is fair to some degree. In *Fig.8* we see the same measurements carried out with the difference being that this time we used the Jaccard index to score the alignments. As it can clearly be seen there is a significant drop-off in alignment scores that is increasingly getting worse towards the lower end of the figure. As elaborated before, this can only be explained by the addition of false positives in the denominator of the Jaccard index (see PSTAR-score $\frac{|A \cap C|}{|C|}$ vs Jaccard index $\frac{|A \cap C|}{|C| + |A \setminus C|}$). However, we expected that the the regression line would flatten out more than it actually did. This means that even after accounting for specificity, there still seems to be a slight correlation between high HomFamRef scores and higher baseline alignment scores. We believe this to be a "probability drift" that is an artifact of slightly shorter HomFamRef alignments causing shorter baseline alignments. These baseline alignments then score higher on average, simply because the chance to do something right by coincidence is higher. For example consider again the case of "adh", which has an exceptionally high-scoring baseline alignment. The HomFam reference alignment contains 5 sequences and there were only 7 gaps inserted overall. Of course, if a random aligner is tasked to place one gap somewhere in each sequence the chances are not bad that the alignment will still score high, even if the gap is not exactly in the right spot. At the very least the baseline should make us aware that any given score is not always and exclusively a result of biologically meaningful alignments.

Figure 8: HomFam protein family MSAs and Jaccard indices of chosen aligners; base-line regression: $y = -0.0005188x + 0.05199$, $R^2 = 0.08261$

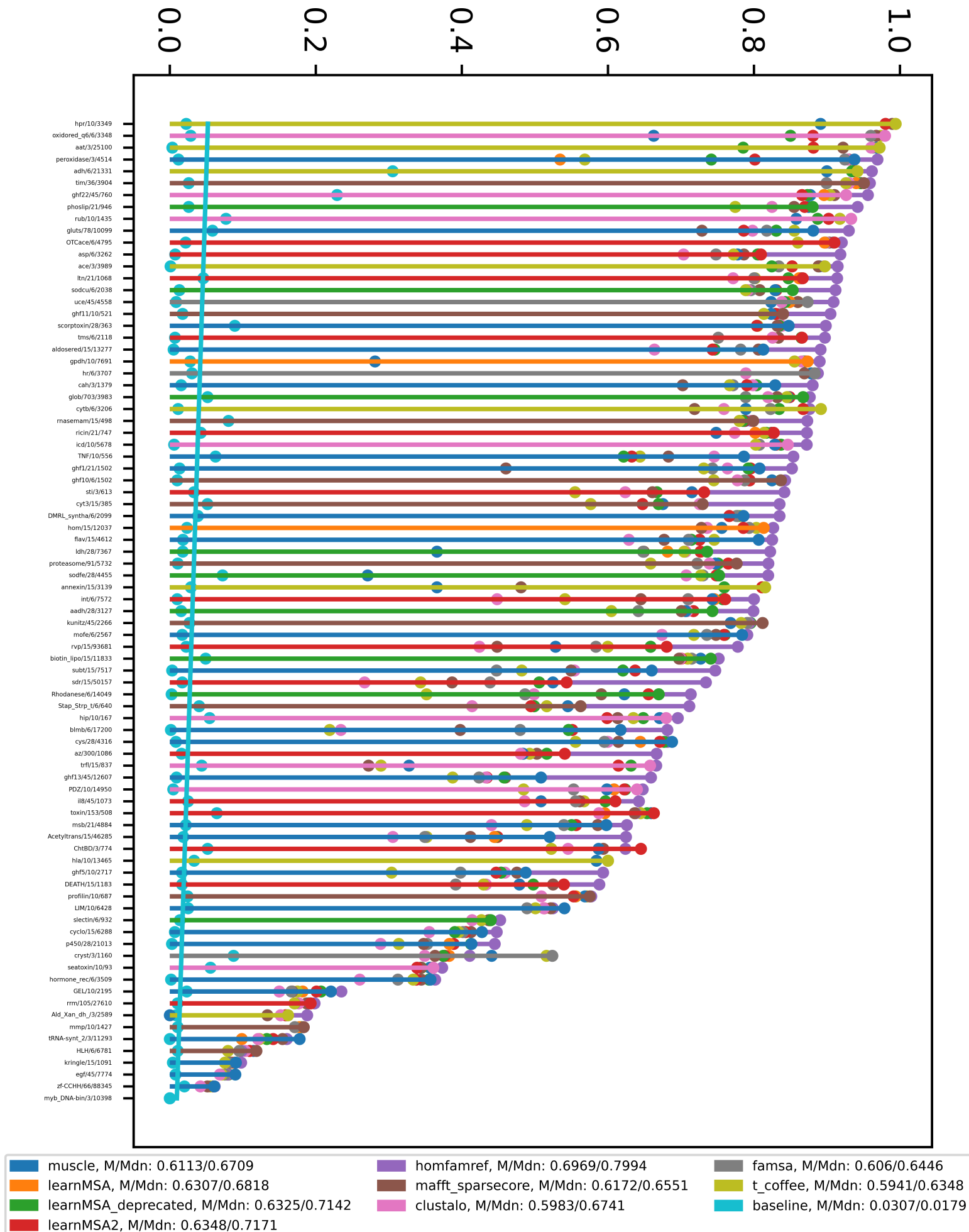


Figure 9: HomFam protein family MSAs and PSTAR-score of chosen aligners without HomFamRef; baseline regression: $y = -0.0004509x + 0.04311$, $R^2 = 0.2658$

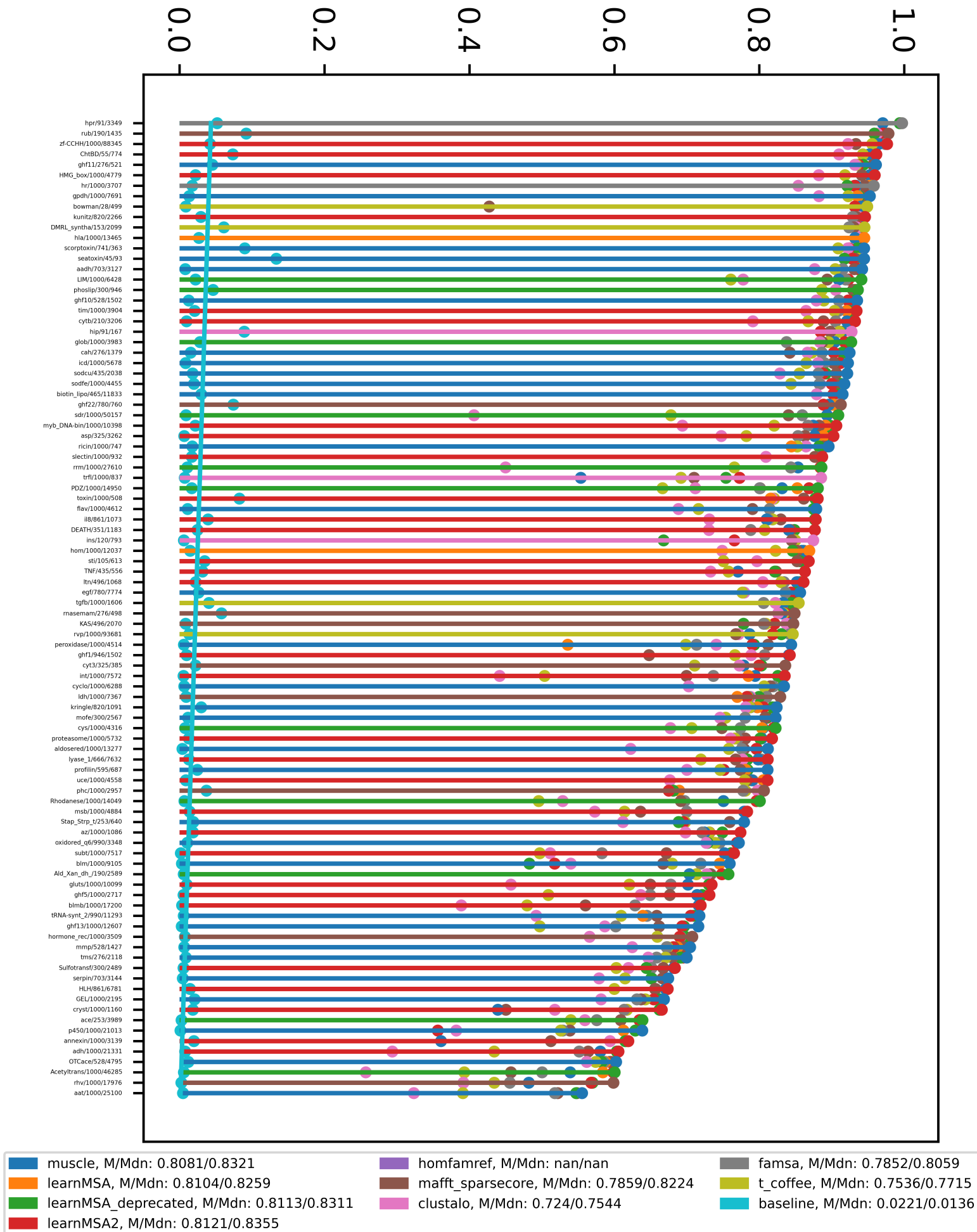
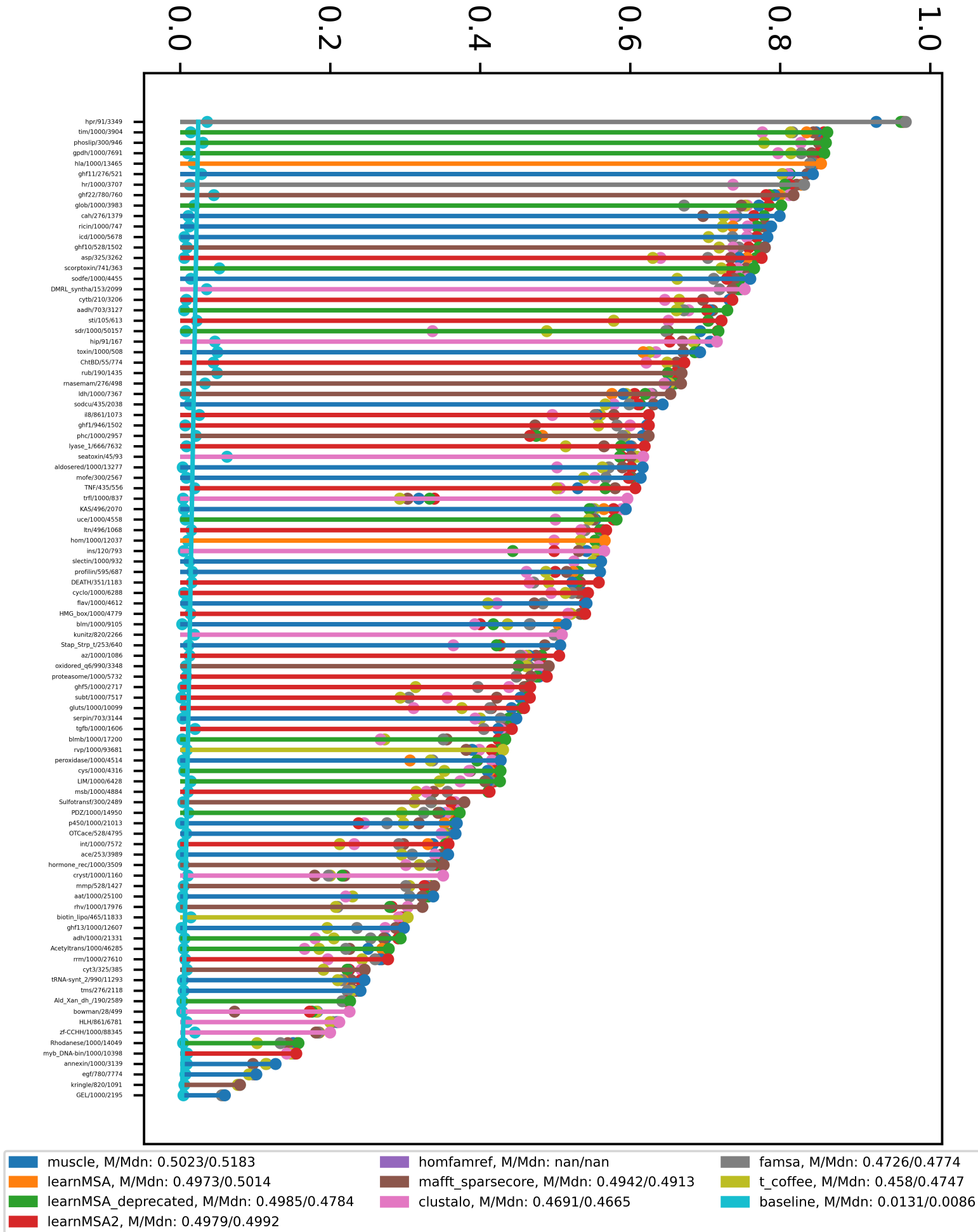


Fig.9 shows the PSTAR-scores for each aligner without simultaneously measuring HomFamRef. Through the inclusion of HomFamRef in the last two rounds of measurements shown in *Fig.7* and *Fig.8* we severely limited data availability to the handful of sequences that had manual alignments for each family. By excluding these references we lower statistical uncertainty of the results. Originally from the legend of *Fig.7* we could see that *learnMSA2* reached the highest score both in median and on average, closely followed by *learnMSA*. Both of these algorithms also were among the most consistent across all protein families with differences between median and mean of around 5 percentage points. The best-performing classic MSA algorithm was *FAMSA*, while also having comparable consistency to *learnMSA*. The worst-performing algorithm was *CLUSTALO*, while also being the least consistent with a difference between median and mean of around 8-9 percentage points. Comparing this to *Fig.9* we can see that *learnMSA2* still scores the highest both in mean and median, again followed by *learnMSA* in the mean but this time *MUSCLE* scores higher in the median, indicating that there are a few unfavourable datasets that *MUSCLE* performs very poorly on. This time *MUSCLE* is also the best scoring classic MSA algorithm both in mean and median. *CLUSTALO* is still the worst both in mean and median. Comparing the regression lines between *Fig.7* and *Fig.9* shows that the R^2 -value has increased by about 23 percentage points, indicating that there are less extreme data points for the baseline, while still maintaining a downwards trend from higher to lower scoring alignments. Both of these things can be explained by a higher average gap count for the shortest alignment across all families, showing that this is not a property inherent to the HomFamRef, but rather to higher scoring alignments, further deepening our suspicions that shorter alignments score better on average to some degree just out of chance. However, it must be noted that part of evaluating an algorithm's quality is also judging its ability to "choose" an appropriate alignment length for a given set of sequences. And clearly some algorithms perform better on this task than others.

In the next step we will first compare how and if the ranking of the algorithms was affected by using the Jaccard index over the PSTAR-score (comparing *Fig.7* and *Fig.8*). This will give us an idea of how the specificities of each aligner compare to each other. Starting with *Fig.8* we can see that *learnMSA2* and *learnMSA* are still ahead by quite a margin both in median and in mean, surprisingly followed in the mean by *MAFFT-sparsecore* and in the median by *CLUSTALO* this time. *MUSCLE* still performs very solid. The last places are occupied by *FAMSA* and *T-COFFEE*.

Figure 10: HomFam protein family MSAs and Jaccard index of chosen aligners without HomFamRef; baseline regression: $y = -0.0002330x + 0.02392$, $R^2 = 0.2413$



As a first thought this could mean that *FAMSA* and *T-COFFEE* have lower specificity than the rest of the aligners. Let us continue by comparing these results to the alignments scored with the Jaccard index without HomFamRef (see *Fig.10*). The flatness and the R^2 of the regression line indicate that both the use of the Jaccard index, as well as the larger data basis are effective means to noise reduction. We believe that this is the most accurate depiction out of all of the shown stem plots when it comes to judging MSA algorithms' abilities to align unknown sequences, which goes to show that there is still a long way to go. Of course the HomFam data set is a "stress test" for MSA algorithms, because the contained sequences were originally meant to be a "distraction" from the real sequences whose SP-score was subsequently measured and compared. Maybe this approach is worth testing in a different, more realistic environment to get more realistic results. According to *Fig.9* *MUSCLE* is now the best scoring MSA algorithm followed by *learnMSA2* and *learnMSA*. *MAFFT-sparsecore* is a close fourth and once again *CLUSTALO*, *FAMSA*, as well as *T-COFFEE* share the last places. This is partly in agreement with [24] in that *learnMSA2*, *learnMSA* and *MUSCLE* are close-by first places, while *T-COFFEE* and *CLUSTALO* do not perform as well. The main difference is that the roles of *MAFFT-sparsecore* and *FAMSA* are exactly reversed. Here *MAFFT* performs notably better than *FAMSA*, instead of the other way around. When comparing this to *Fig.7* we can see that originally, when we had a small data basis and used the PSTAR-score that *FAMSA* performed better than *MAFFT* and even *MUSCLE*. This early lead dwindled when increasing the size of the data basis, where it had already fallen behind *MUSCLE* (see *Fig.9*). When considering the specificity by using the Jaccard index for scoring it was already behind *MUSCLE* and *MAFFT* (see *Fig.8*). Of course, when combining both circumstances in *Fig.10* we find that *FAMSA* lost its lead over *MUSCLE* and *MAFFT*, although still performing better than *CLUSTALO* and *T-COFFEE*. Overall, this leads us to believe that *FAMSA* is a bit of a paper tiger and a result of a small reference data basis and low specificity. However, we should also bear in mind that the tested MSA algorithms all aim to maximize the SP-score during alignment of the sequences. Therefore we should take care when trying to evaluate their overall quality, since the low Jaccard index values might as well be more of a measure whether the SP-score is actually a sensible thing to optimize during MSA generation. Additionally we showed before that our approach does not have perfect sensitivity or specificity due to the fact that there are regions in the optimal MSA solution set that are not classifiable by PSTAR. It might very well be that *FAMSA* heeds transitive residue equivalence constraints or something else that is not classifiable by PSTAR just a bit more than the other aligners and we might be judging it unfairly for that. As others have pointed out, another alternative explanation for the good results of HOMSTRAD might be that "HOMSTRAD emphasizes pair-wise alignments of

globally alignable proteins, while the Balibase BB subset comprises more challenging cases requiring alignment of many proteins where homologous segments are surrounded by highly diverged and/or unrelated sequence” and ”We regard HOMSTRAD as a less stringent test given that most alignments are pair-wise and all proteins have the same domain organization.” [44], which is especially amusing considering that *MUSCLE* scored best in our pairwise alignment based benchmarking (and thus by extension produced the most similar results to HOMSTRAD). This also raises the question of how BaliFam would perform in comparison in future tests.

3.2 Measuring specificity and overall uncertainty

In the following we will be taking a closer look at the differences between measuring MSAs with both the Jaccard index, as well as the PSTAR-score.

MSA algorithm	PSTAR-score	Jaccard index	Difference	% Difference
HomFamRef	0.9000	0.6969	0.2031	22.57
learnMSA2	0.8483	0.6348	0.2135	25.17
learnMSA	0.8420	0.6307	0.2113	25.10
learnMSA_depr	0.8459	0.6325	0.2134	25.23
MUSCLE	0.8044	0.6113	0.1931	24.01
MAFFT-sparsecore	0.8083	0.6172	0.1911	23.64
FAMSA	0.8210	0.6060	0.2150	26.19
CLUSTALO	0.7739	0.5983	0.1756	22.69
T-COFFEE	0.7985	0.5941	0.2044	25.60
Baseline	0.0594	0.0307	0.0287	48.32

Table 1: Overview of differences between mean scores of *Fig.7* (PSTAR-score) and *Fig.8* (Jaccard index). The last column contains the percentage of the difference of the original score (PSTAR-score).

If we now assume that the percentage of difference of the PSTAR-score and the Jaccard index is proportional to the number of false positives of each aligner, i.e. inversely proportional to the specificity, which we have good reason to assume, then we get that HomFamRef has the best sensitivity and also the best specificity of all of the aligners (see *Tab.1*). The baseline has the worst sensitivity and also the worst specificity. Let us now see, if these results are consolidated in the larger data set without HomFamRef.

Comparing *Tab.1* and *Tab.2*, we can see that *CLUSTALO* has still the best specificity, followed by *MAFFT-sparsecore* and then *MUSCLE*, *learnMSA_deprecated*, *learnMSA*, *learnMSA2* very close-by, followed by *T-COFFEE* and then finally *FAMSA*. In *Tab.1* *learnMSA2*, *learnMSA* and *learnMSA_deprecated* are shuffled up slightly. We assume *Tab.2* to represent reality better, since the data basis is larger. As an interesting side result we can see a slight drift of *learnMSA* algorithms toward lower specificity over time. We can not reliably judge, if this is a significant result. However, it might be something to watch out for in future versions of the aligner. Note that low Jaccard index values technically only mean that both sets have little agreement, which can also mean that our reference is not great at modeling the problem of MSA with the SP-score. On one hand, most common reference databases are also structure-based and in the case

of HOMSTRAD for example they are even partly based on pairwise structural alignments (see the end of the previous section). Hence, deviations that are the results of pairwise structure alignments should also be noticeable in benchmark tests that were performed with these databases. Meaning that our approach is comparable in that regard. However, there is one key difference in our approach, which is that we use DALI to generate the pairwise reference alignments. According to table 2 in [31], which was briefly mentioned earlier, DALI performs worst out of all tested structure aligners on the HOMSTRAD test set in terms of " f_{car} ", a sensitivity measure. Of course this lowers the overlap between our DALI-based reference and HOMSTRAD. This means in theory that, if we were to score the test sets *CDD* [45] or *CATH* [46], which DALI ranked highest in, we should see an overall upward shift in specificity values. It also should mean that if we were to base our reference on *UniAlign* [47], which scored highest on the HOMSTRAD test set, we should also be able to observe the same effect.

MSA algorithm	PSTAR-score	Jaccard index	Difference	% Difference
learnMSA2	0.8121	0.4979	0.3142	38.70
learnMSA	0.8104	0.4973	0.3131	38.64
learnMSA_depr	0.8113	0.4985	0.3128	38.56
MUSCLE	0.8081	0.5023	0.3058	37.84
MAFFT-sparsecore	0.7859	0.4942	0.2917	37.12
FAMSA	0.7852	0.4726	0.3126	39.81
CLUSTALO	0.7240	0.4691	0.2549	35.21
T-COFFEE	0.7536	0.4580	0.2956	39.23
Baseline	0.0221	0.0131	0.009	42.76±2.04

Table 2: Overview of differences between mean scores of *Fig.9* (PSTAR-score) and *Fig.10* (Jaccard index) **without HomFamRef**. The last column contains the percentage of the difference of the original score (PSTAR-score).

4 Conclusion

We developed a software that is capable of judging any MSA algorithm based on its output and to compare the quality of the alignments with newly-developed measure indices, given any common subset of sequences of the outputs that are also exactly contained within an experimentally-obtained protein structure. This was done by using the tertiary structure of proteins and by using DALI to obtain pairwise structure alignments that served as a reference. We made the assumption that the SP-score as well as its pairwise structural alignment counterpart,

the PSTAR-score, are not sufficient to fully and fairly judge the quality of MSA algorithms and we could not disprove this assumption with the data that we acquired. We found a remarkable difference between PSTAR-score and Jaccard index when judging alignment quality that we could not explain by anything different than a difference in specificity of the different alignments. This might partly be a result of heuristic MSA algorithms being designed to maximize the SP-score. We comprehensively showed on the example of *FAMSA* and *MAFFT-sparsecore* that both how we measure alignment quality and how much data is available for measurement can have a severe impact on the final ranking of different MSA algorithms. We showed that the *HomFamRef* on average had the highest sensitivity among all of the aligners that were tested here and, given our assumption about the significance of the specificity is correct, we also showed that it has the highest specificity. We conclude that the *HomFamRef* is a reliable, but too limited reference for judging MSA quality. This is backed by our findings that increasing data availability without changing the scoring measure already had an impact on the final ranking of the different algorithms. We find that *learnMSA2* is an MSA algorithm with good sensitivity and average specificity and it ranks the best among all of the algorithms when using the PSTAR-score and is only very tightly surpassed by *MUSCLE* when using the Jaccard index to measure alignment quality. We acknowledged that our approach has systematic flaws, which lend it only suboptimal capabilities to correctly identify true positives, inevitably lowering its own specificity and sensitivity. However, emboldened by our results, we feel that it still possesses a fine enough granularity to make it a viable and reliable measure to fairly judge MSA quality nonetheless. For future research on the matter we encourage the development of MSA algorithms that take into consideration the specificity of the solution, i.e. that do not simply estimate the optimization problem of MSA with the SP-score, but instead also utilize measures like the Jaccard index to generate their output. This is partly because we feel like the SP-score has become a bit of a positive feedback echo chamber, because we do not know how to explain the differences between Jaccard index and PSTAR-score, if we were to assume that sensitivity and specificity are effectively the same for MSAs. If we had compelling evidence that this is the case, we could argue that the lack of specificity and its corresponding measures could actually be due to the lack of specificity of PSTAR-MSA itself. Either way this would be a good thing, since we feel like knowing what we do not know is more important than the acquisition of "false positive" results. Regarding the future of PSTAR-MSA the next steps would be to use the *AlphaFold* database to evaluate alignments based on structure prediction and to observe how the results compare to the ones we presented here. Furthermore, the implementation of other pairwise or multiple structure aligners should be a goal to pursue, since all of this helps to consolidate or disperse confidence in the results.

5 Side results

5.1 The Hamming distance defined in terms of our set representations

The Hamming distance [48] is a common way to measure the similarity of two character strings. We define it as follows:

Definition 13. : Hamming distance

Given two strings $S_i, S_k \in (\Sigma \cup \{-\})^M$ for some alphabet Σ and with $S_i = (s_{i1}, \dots, s_{ij}, \dots, s_{iM})$ and fixed $i, k \in \{1, \dots, N\} = I$, as well as $j \in \{1, \dots, M\} = J$. Let $D := \{((i, j), (k, j)) \mid \forall j \in J : s_{ij} \sim_A s_{kj}\}$ be the set of similar pairs for some equivalence relation $\sim_A$. Then,

$$d_H(S_i, S_k) := M - |D|$$

is called the **Hamming distance** between S_i and S_k .

Since we customized the notation of the Hamming distance with the introduction of the similar pair set given some general equivalence relation, let us show that it is still a metric.

Proposition 1. $((\Sigma \cup \{-\})^M, d_H)$ is a metric space.

Proof. Given $S_i, S_k, S_l \in (\Sigma \cup \{-\})^M$ for arbitrary fixed $i, k, l \in \{1, \dots, N\}$.

Obviously $d_H(S_i, S_i) = M - |\{((i, j), (i, j)) \mid \forall j \in \{1, \dots, M\}\}|$, because of the reflexivity of $\sim_A$ we have $s_{ij} \sim_A s_{ij}$. Thus $d_H(S_i, S_i) = M - M = 0$.

Positivity:

$S_i \neq S_k \implies \exists j \in \{1, \dots, M\} : (s_{ij}, s_{kj}) \notin \sim_A \implies |D| < M \implies d_H(S_i, S_k) > 0$

Symmetry:

Since $\sim_A$ is symmetric, we have $s_{ij} \sim_A s_{kj} \implies s_{kj} \sim_A s_{ij}$. Thus for every j , where $((i, j), (k, j)) \in D$ we also have $((k, j), (i, j)) \in D'$, which implies that $|D| = |D'|$.

Triangle inequality:

Given $d_H(S_i, S_k) = x$ (1), $d_H(S_k, S_l) = y$ (2) and $d_H(S_i, S_l) = z$ (3). We proceed to show $z \leq x + y$.

- (1) $\implies \exists J_x \subseteq J$ with $|J_x| = M - x$ and $\forall j \in J_x : s_{ij} \sim_A s_{kj}$
- (2) $\implies \exists J_y \subseteq J$ with $|J_y| = M - y$ and $\forall j \in J_y : s_{kj} \sim_A s_{lj}$
- (3) $\implies \exists J_z \subseteq J$ with $|J_z| = M - z$ and $\forall j \in J_z : s_{ij} \sim_A s_{lj}$

We distinguish between 2 cases:

Case 1: $J_x \cap J_y = \emptyset$

$$\min(x + y) = \min(2M - (|J_x| + |J_y|)) = \min(2M - \max(|J_x| + |J_y|)) = M = \max(M - \min(|J_z|)) = \max(z)$$

Case 2: $J_x \cap J_y \neq \emptyset$

Let $X := J_x \cap J_y$, then $\forall j \in X : s_{ij} \sim_A s_{kj}$ and $s_{kj} \sim_A s_{lj}$. Because of the transitivity of $\sim_A$, we therefore also have $s_{ij} \sim_A s_{lj}$ and thus $j \in X \implies j \in J_z$. Furthermore $x + y = 2M - (|J_x \setminus X| + |J_y \setminus X| + 2|X|) \geq M - (|J_z \setminus X| + |X|) = z$. For a proof by contradiction we assume:

$$\begin{aligned} 2M - (|J_x \setminus X| + |J_y \setminus X| + 2|X|) &< M - (|J_z \setminus X| + |X|) \\ \iff -(|J_x \setminus X| + |J_y \setminus X| + 2|X|) &< -M - (|J_z \setminus X| + |X|) \\ \iff |J_x \setminus X| + |J_y \setminus X| + 2|X| &> M + |J_z \setminus X| + |X| \\ \iff |J_x \setminus X| + |J_y \setminus X| + |X| &> M + |J_z \setminus X| \end{aligned}$$

$$\iff \underbrace{|J_x \setminus X| + |J_y \setminus X| + |X|}_{\leq M} > M$$

This contradicts the assumption and hence $z \leq x + y$ □

5.2 A small conjecture about the uniqueness of the PSTAR reference set representation

Conjecture 1. *For a fixed sequence database and fixed comparison strategy $J \neq \emptyset$ and if the structure aligner gives the same output for the same input, then for each set of unique sequences $C = \{C_1, \dots, C_i, \dots, C_N\}$ there is at most one $P_{ref, C}^u$.*

5.3 A metric space involving three sets and a corollary

Theorem 1. *For an arbitrary set C and collection of sets X , as well as sets $A, B \in X$ and*

$$RSM_C(A, B) := \frac{1}{2} \left(\left| \frac{|A \cap C|}{|A \cup C|} - \frac{|B \cap C|}{|B \cup C|} \right| + 1 - \frac{|A \cap B|}{|A \cup B|} \right)$$

*(X, RSM_C) is a **metric space**. We define $RSM_C(A, B) := 0$, if $A = B = \emptyset$ and $RSM_C(A, B) := \frac{1}{2}$, if $A = C = \emptyset$ or $B = C = \emptyset$. We call $RSM_C(A, B)$ the **reference set metric** for reference set C .*

Proof. (1) **Identity of indiscernibles:**

$$A = B \implies \frac{1}{2} * \left(\left| \frac{|A \cap C|}{|A \cup C|} - \frac{|A \cap C|}{|A \cup C|} \right| + 1 - \frac{|A|}{|A|} \right) = \frac{1}{2} \left(\left| \frac{0}{|A \cup C|} \right| + 1 - 1 \right) = 0$$

(2) **Symmetry:**

$$\begin{aligned}
 RSM_C(A, B) &= \frac{1}{2} * \left(\left| \frac{|A \cap C|}{|A \cup C|} - \frac{|B \cap C|}{|B \cup C|} \right| + 1 - \frac{|A \cap B|}{|A \cup B|} \right) \\
 &= \frac{1}{2} * \left(\left| - \left(\frac{|B \cap C|}{|B \cup C|} - \frac{|A \cap C|}{|A \cup C|} \right) \right| + 1 - \frac{|B \cap A|}{|B \cup A|} \right) \\
 &= RSM_C(B, A)
 \end{aligned}$$

(3) **Positivity:** We know that for $A \neq B \implies J(A, B) > 0 \implies 1 - \frac{|A \cap B|}{|A \cup B|} > 0$ and $\left| \frac{|A \cap C|}{|A \cup C|} - \frac{|B \cap C|}{|B \cup C|} \right| \geq 0$ and thus $RSM_C(A, B) > 0$

(4) **Triangle inequality:** Let $A, B, D \in X$, then

$$\begin{aligned}
 RSM_C(A, B) + RSM_C(B, D) &\geq RSM_C(A, D) \\
 \iff \\
 \left| \frac{|A \cap C|}{|A \cup C|} - \frac{|B \cap C|}{|B \cup C|} \right| + \left| \frac{|B \cap C|}{|B \cup C|} - \frac{|D \cap C|}{|D \cup C|} \right| - \left| \frac{|A \cap C|}{|A \cup C|} - \frac{|D \cap C|}{|D \cup C|} \right| \\
 + \frac{|A \cap D|}{|A \cup D|} - \frac{|A \cap B|}{|A \cup B|} - \frac{|B \cap D|}{|B \cup D|} &\geq -1
 \end{aligned}$$

We rename $a := \frac{|A \cap C|}{|A \cup C|}$, $b := \frac{|B \cap C|}{|B \cup C|}$, $d := \frac{|D \cap C|}{|D \cup C|}$, $f := \frac{|A \cap D|}{|A \cup D|}$, $g := \frac{|A \cap B|}{|A \cup B|}$, $h := \frac{|B \cap D|}{|B \cup D|}$ and get

$$|a - b| + |b - d| - |a - d| + f - g - h \geq -1$$

We proceed to distinguish between 6 cases:

- 1) $a \geq b \geq d \implies f - g - h \geq -1$
- 2) $a \geq d \geq b \implies -2b + 2d + f - g - h \geq -1$
- 3) $b \geq a \geq d \implies -2a + 2b + f - g - h \geq -1$
- 4) $b \geq d \geq a \implies -2d + 2b + f - g - h \geq -1$
- 5) $d \geq a \geq b \implies -2b + 2a + f - g - h \geq -1$
- 6) $d \geq b \geq a \implies f - g - h \geq -1$

We will have a look at the first case and see that

$$f - g - h \geq -1 \iff 2 - \frac{|A \cap B|}{|A \cup B|} - \frac{|B \cap D|}{|B \cup D|} \geq 1 - \frac{|A \cap D|}{|A \cup D|} \iff J(A, B) + J(B, D) \geq J(A, D)$$

which is the triangle inequality for the Jaccard metric.[35] For cases 2) through 5) we can see that the smaller of the two relevant variables is always associated with the negative coefficient. This means that the sum of the first two terms in those cases are always greater than zero or zero, if both involved variables are equal. $\square$

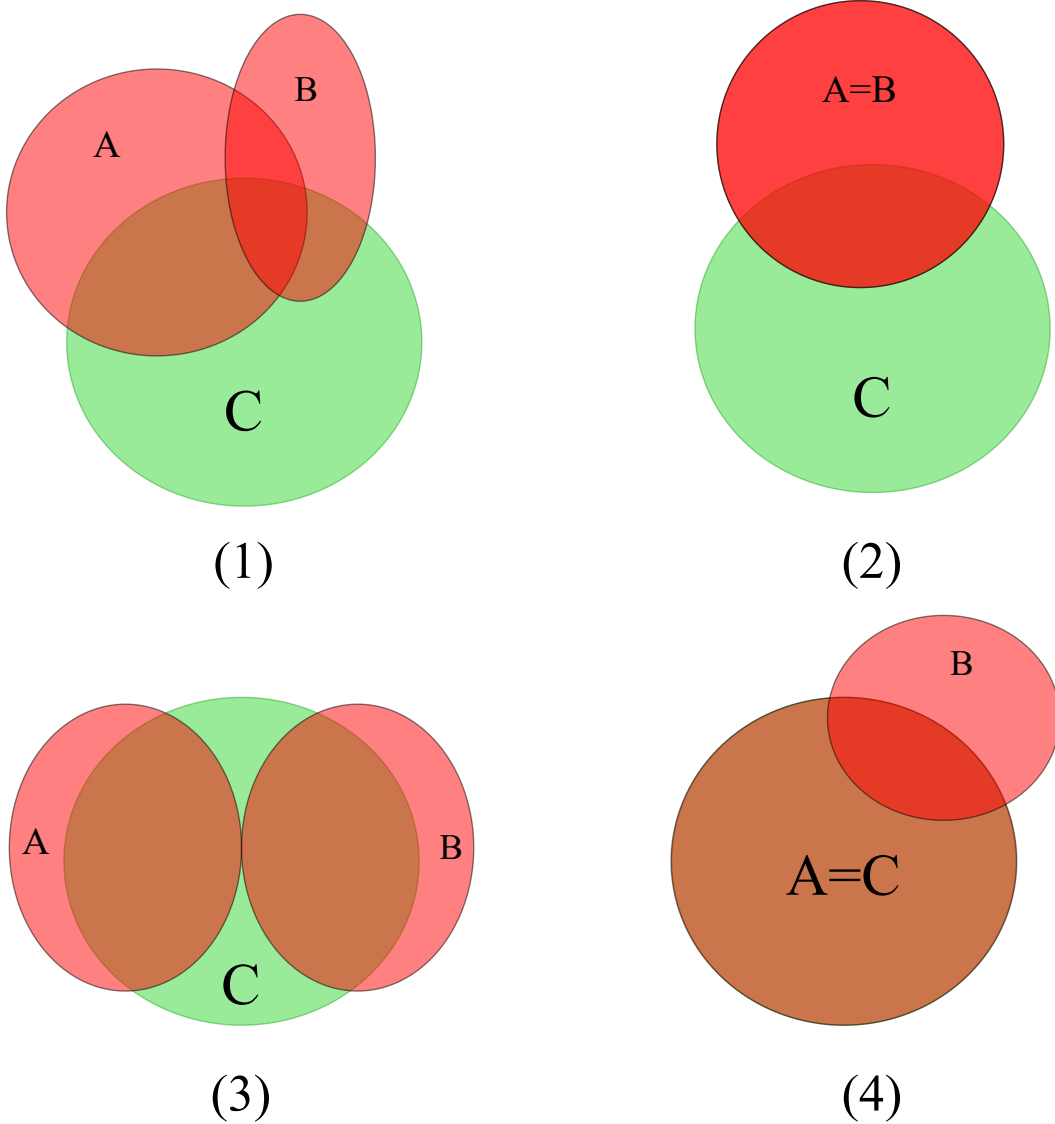


Figure 11: Example cases of the relative position of three different sets and approximate values for $RSM_C(A, B)$. We arbitrarily set $|C| := 100$. **Case (1):** From the looks of it let $|A| := 75$ and $|B| := 30$. Then $RSM_C(A, B) = \frac{1}{2} \left(\left| \frac{37}{138} - \frac{10}{120} \right| + 1 - \frac{9}{96} \right) \approx 0.54552$. **Case (2):** $RSM_C(A, B) = \frac{1}{2} (0 + 1 - 1) = 0$. **Case (3):** $RSM_C(A, B) = \frac{1}{2} (0 + 1 - 0) = 0.5$. **Case (4):** $RSM_C(A, B) = \frac{1}{2} \left(\left| \frac{100}{100} - \frac{14}{128} \right| + 1 - \frac{14}{128} \right) \approx 0.89063$

Corollary 1.

(1) $J(A, C) = J(B, C) \implies \max(RSM_C(A, B)) = \frac{1}{2} \quad \forall A, B \in X \text{ and arbitrary } C.$

That's because $J(A, C) = J(B, C) \iff \left| \frac{|A \cap C|}{|A \cup C|} - \frac{|B \cap C|}{|B \cup C|} \right| = 0$

$\implies \max(\frac{1}{2}(0 + J(A, B))) = 0$

(2) $A = C \implies \frac{1}{2} * \left(\left| \frac{|A|}{|A|} - \frac{|B \cap A|}{|B \cup A|} \right| + 1 - \frac{|A \cap B|}{|A \cup B|} \right) = 1 - \frac{A \cap B}{A \cup B} = J(A, B)$

And $B = C \implies RS M_C(A, B) = J(A, B)$

(3) $A \neq B \not\Rightarrow \left| \frac{|A \cap C|}{|A \cup C|} - \frac{|B \cap C|}{|B \cup C|} \right| > 0$. Thus, $\left| \frac{|A \cap C|}{|A \cup C|} - \frac{|B \cap C|}{|B \cup C|} \right|$ is not a metric.

To see this suppose sets $A \neq B$ and C , but with $|A| = |B|$, e.g. $A = \{1, 2, 3, 4\}$, $B = \{5, 6, 7, 8\}$ and $C = \{3, 4, 5, 6\}$, then $|A \cup C| = |B \cup C| = |\{1, 2, 3, 4, 5, 6\}| = |\{3, 4, 5, 6, 7, 8\}|$, but at the same time $|A \cap C| = |B \cap C| = |\{3, 4\}| = |\{5, 6\}|$. Then we get $\left| \frac{2}{6} - \frac{2}{6} \right| = 0$, even though $A \neq B$.

(4) From the proof of the triangle inequality we can see that $\left| \frac{|A \cap C|}{|A \cup C|} - \frac{|B \cap C|}{|B \cup C|} \right| + \left| \frac{|B \cap C|}{|B \cup C|} - \frac{|D \cap C|}{|D \cup C|} \right| - \left| \frac{|A \cap C|}{|A \cup C|} - \frac{|D \cap C|}{|D \cup C|} \right| \geq 0$. Together with the semi-positivity from (3) this means that $RS M_C(A, B) - 1 + J(A, B) = \left| \frac{|A \cap C|}{|A \cup C|} - \frac{|B \cap C|}{|B \cup C|} \right|$ is a pseudo-metric.

5.4 An unproven metric involving three sets

Conjecture 2. Combined dissimilarity measure

$$CDM(A, B)_C := 1 - \frac{2}{3} * \frac{|A \cap C \cap B| + |A \cap C| + |B \cap C|}{\frac{|A| + |B|}{2} + |C| + |A \cup C \cup B| - |A \cap C \cap B|}$$

is called the **combined dissimilarity measure**. Let S be the collection of all sets. For all $A, B \in S$ and each constant set C , $(S, CDM(A, B)_C)$ is a metric space.

Remark. This was discovered while searching for an appropriate distance measure between MSAs. However, proving the triangle inequality for this was not really worth it and was given up after a few days, because this measure does not measure distance in a way we want. It has some interesting (assumed) properties, including $CDM(A, B)_C = 0 \iff A = B = C$. This is because it was designed in a way that whenever $A = B$ or $A = C$ or $B = C$ it collapses into the regular Jaccard-metric for the remaining two sets. It changes the weights of the individual set intersections. The intersections $A \cap B \cap C$, $A \cap C$ and $B \cap C$ are weighted more heavily than the intersection $A \cap B$, although latter also decreases the distance between sets A and B . Just not as drastic as the former intersections. As to what this is good for, we can only guess.

References

- [1] Robert C Edgar and Serafim Batzoglou. Multiple sequence alignment. *Current opinion in structural biology*, 16(3):368–373, 2006.

- [2] Rakesh Trivedi and Hampapathalu Adimurthy Nagarajaram. Substitution scoring matrices for proteins-an overview. *Protein Science*, 29(11):2150–2163, 2020.
- [3] Isaac Elias. Settling the intractability of multiple alignment. *Journal of Computational Biology*, 13(7):1323–1339, 2006.
- [4] Winfried Just. Computational complexity of multiple sequence alignment with sp-score. *Journal of computational biology*, 8(6):615–623, 2001.
- [5] Kenji Mizuguchi, Charlotte M Deane, Tom L Blundell, and John P Overington. Homstrad: a database of protein structure alignments for homologous families. *Protein science*, 7(11):2469–2471, 1998.
- [6] Andrej Sali and Tom L Blundell. Definition of general topological equivalence in protein structures: A procedure involving comparison of properties and relationships through simulated annealing and dynamic programming. *Journal of molecular biology*, 212(2):403–428, 1990.
- [7] Stefano Pascarella and Patrick Argos. A data bank merging related protein structures and sequences. *Protein Engineering, Design and Selection*, 5(2):121–137, 1992.
- [8] Michael J Sutcliffe, I Haneef, D Carney, and TL Blundell. Knowledge based modelling of homologous proteins, part i: Three-dimensional frameworks derived from the simultaneous superposition of multiple structures. *Protein Engineering, Design and Selection*, 1(5):377–384, 1987.
- [9] Lorenzo Pantolini, Gabriel Studer, Joana Pereira, Janani Durairaj, Gerardo Tauriello, and Torsten Schwede. Embedding-based alignment: combining protein language models with dynamic programming alignment to detect structural similarities in the twilight-zone. *Bioinformatics*, 40(1):btad786, 2024.
- [10] Julie D. Thompson, Frédéric Plewniak, and Olivier Poch. Balibase: a benchmark alignment database for the evaluation of multiple alignment programs. *Bioinformatics (Oxford, England)*, 15(1):87–88, 1999.
- [11] Bharath Reddy and Richard Fields. Performance analysis of multiple sequence alignment tools. In *Proceedings of the 2024 ACM Southeast Conference*, pages 167–174, 2024.
- [12] Fabiano Sviatopolk-Mirsky Pais, Patrícia de Cássia Ruy, Guilherme Oliveira, and Roney Santos Coimbra. Assessing the efficiency of multiple

sequence alignment programs. *Algorithms for molecular biology*, 9:1–8, 2014.

- [13] Mihaly Varadi, Damian Bertoni, Paulyna Magana, Urmila Paramval, Ivanna Pidruchna, Malarvizhi Radhakrishnan, Maxim Tsenkov, Sreenath Nair, Milot Mirdita, Jingi Yeo, et al. Alphafold protein structure database in 2024: providing structure coverage for over 214 million protein sequences. *Nucleic acids research*, 52(D1):D368–D375, 2024.
- [14] Jianzhi Zhang and Sudhir Kumar. Detection of convergent and parallel evolution at the amino acid sequence level. *Molecular biology and evolution*, 14(5):527–536, 1997.
- [15] Walter M Fitch. Distinguishing homologous from analogous proteins. *Systematic zoology*, 19(2):99–113, 1970.
- [16] Robert B Russell, Mansoor AS Saqi, Roger A Sayle, Paul A Bates, and Michael JE Sternberg. Recognition of analogous and homologous protein folds: analysis of sequence and structure conservation. *Journal of molecular biology*, 269(3):423–439, 1997.
- [17] Fabian Sievers, Andreas Wilm, David Dineen, Toby J Gibson, Kevin Karplus, Weizhong Li, Rodrigo Lopez, Hamish McWilliam, Michael Remmert, Johannes Söding, et al. Fast, scalable generation of high-quality protein multiple sequence alignments using clustal omega. *Molecular systems biology*, 7(1):539, 2011.
- [18] Da-Fei Feng and Russell F Doolittle. Progressive sequence alignment as a prerequisite to correct phylogenetic trees. *Journal of molecular evolution*, 25:351–360, 1987.
- [19] Robert C Edgar. Muscle: multiple sequence alignment with high accuracy and high throughput. *Nucleic acids research*, 32(5):1792–1797, 2004.
- [20] Kazunori D Yamada, Kentaro Tomii, and Kazutaka Katoh. Application of the MAFFT sequence alignment program to large data—reexamination of the usefulness of chained guide trees. *Bioinformatics*, 32(21):3246–3251, 2016.
- [21] Sebastian Deorowicz, Agnieszka Debudaj-Grabysz, and Adam Gudyś. Famsa: Fast and accurate multiple sequence alignment of huge protein families. *Scientific reports*, 6(1):33964, 2016.

- [22] Cédric Notredame, Desmond G Higgins, and Jaap Heringa. T-coffee: A novel method for fast and accurate multiple sequence alignment. *Journal of molecular biology*, 302(1):205–217, 2000.
- [23] Felix Becker and Mario Stanke. learnMSA: learning and aligning large protein families. *GigaScience*, 11:giac104, 2022.
- [24] Felix Becker and Mario Stanke. learnMSA2: deep protein multiple alignments with large language and hidden Markov models. *Bioinformatics*, 40(Supplement_2):ii79–ii86, 2024.
- [25] Rachel Kolodny and Nathan Linial. Approximate protein structural alignment in polynomial time. *Proceedings of the National Academy of Sciences*, 101(33):12201–12206, 2004.
- [26] FR Salemme, Michael D Miller, and Steven R Jordan. Structural convergence during protein evolution. *Proceedings of the National Academy of Sciences*, 74(7):2820–2824, 1977.
- [27] Gabriele Ausiello, Daniele Peluso, Allegra Via, and Manuela Helmer-Citterich. Local comparison of protein structures highlights cases of convergent evolution in analogous functional sites. *BMC bioinformatics*, 8:1–9, 2007.
- [28] Cyrus Chothia and Arthur M Lesk. The relation between the divergence of sequence and structure in proteins. *The EMBO journal*, 5(4):823–826, 1986.
- [29] Jianzhu Ma and Sheng Wang. Algorithms, applications, and challenges of protein structure alignment. *Advances in Protein Chemistry and Structural Biology*, 94:121–175, 2014.
- [30] Liisa Holm and Chris Sander. Protein structure comparison by alignment of distance matrices. *Journal of molecular biology*, 233(1):123–138, 1993.
- [31] Liisa Holm. Dali and the persistence of protein shape. *Protein Science*, 29(1):128–140, 2020.
- [32] Benjamin P Blackburne and Simon Whelan. Measuring the distance between multiple sequence alignments. *Bioinformatics*, 28(4):495–502, 2012.
- [33] Julie D Thompson, Frédéric Plewniak, and Olivier Poch. A comprehensive comparison of multiple sequence alignment programs. *Nucleic acids research*, 27(13):2682–2690, 1999.

- [34] William R Pearson. Selecting the right similarity-scoring matrix. *Current protocols in bioinformatics*, 43(1):3–5, 2013.
- [35] Sven Kosub. A note on the triangle inequality for the Jaccard distance. *Pattern Recognition Letters*, 120:36–38, 2019.
- [36] Allan David Gordon. *Classification*. CRC Press, 1999.
- [37] Douglas G Altman and J Martin Bland. Diagnostic tests. 1: Sensitivity and specificity. *BMJ: British Medical Journal*, 308(6943):1552, 1994.
- [38] Protein Data Bank. Protein data bank. *Nature New Biol*, 233(223):10–1038, 1971.
- [39] Brad Chapman and Jeffrey Chang. Biopython: Python tools for computational biology. *ACM Sigbio Newsletter*, 20(2):15–19, 2000.
- [40] Benjamin Buchfink, Klaus Reuter, and Hajk-Georg Drost. Sensitive protein alignments at tree-of-life scale using diamond. *Nature methods*, 18(4):366–368, 2021.
- [41] Gordon Blackshields, Fabian Sievers, Weifeng Shi, Andreas Wilm, and Desmond G Higgins. Sequence embedding for fast construction of guide trees for multiple sequence alignment. *Algorithms for Molecular Biology*, 5:1–11, 2010.
- [42] Kieran Boyce, Fabian Sievers, and Desmond G Higgins. Instability in progressive multiple sequence alignment algorithms. *Algorithms for molecular biology*, 10:1–10, 2015.
- [43] Edgar Gabriel, Graham E Fagg, George Bosilca, Thara Angskun, Jack J Dongarra, Jeffrey M Squyres, Vishal Sahay, Prabhanjan Kambadur, Brian Barrett, Andrew Lumsdaine, et al. Open MPI: Goals, concept, and design of a next generation mpi implementation. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface: 11th European PVM/MPI Users’ Group Meeting Budapest, Hungary, September 19-22, 2004. Proceedings 11*, pages 97–104. Springer, 2004.
- [44] Robert C Edgar and Igor Tolstoy. Muscle-3d: scalable multiple protein structure alignment. *bioRxiv*, pages 2024–10, 2024.
- [45] Aron Marchler-Bauer, Myra K Derbyshire, Noreen R Gonzales, Shennan Lu, Farideh Chitsaz, Lewis Y Geer, Renata C Geer, Jane He, Marc Gwadz, David I Hurwitz, et al. Cdd: Ncbi’s conserved domain database. *Nucleic acids research*, 43(D1):D222–D226, 2015.

- [46] Christine A Orengo, Alex D Michie, Susan Jones, David T Jones, Mark B Swindells, and Janet M Thornton. Cath—a hierarchic classification of protein domain structures. *Structure*, 5(8):1093–1109, 1997.
- [47] Chunyu Zhao and Ahmet Sacan. Unalign: protein structure alignment meets evolution. *Bioinformatics*, 31(19):3139–3146, 2015.
- [48] Abraham Bookstein, Vladimir A Kulyukin, and Timo Raita. Generalized hamming distance. *Information Retrieval*, 5:353–375, 2002.